

# Алгоритми

Довідковий каталог **алгоритмів цифрової обробки сигналів (DSP), математичних, графічних та сигнально-обробних алгоритмів**, реалізованих у чотирьох вимірювальних модулях Phonyser — **Генераторі сигналів, Осцилографі, FFT-аналізаторі** та поданні **Частотної характеристики**. Для кожного алгоритму подано короткий опис того, *що він робить і як використовується у застосунку*, з посиланнями `file:line` на вихідний код у стані на момент створення цього документа.

Документ згенеровано читанням коду та його коментарів; він описує те, що **фактично реалізовано** (включно зі шляхами, які підключені, але наразі вимкнені — вони позначені окремо), а не задуманий дизайн.

## Показчик іменованих алгоритмів

Епонімні / підручкові алгоритми, що використані в коді, із зазначенням, де читати докладніше. Детальні розділи за модулями нижче містять математику та посилання `file:line`.

### Синтез сигналу та квантування

| Алгоритм                                         | Що це таке                                                                                                                                                                                     | Де              |
|--------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| <b>Пряма цифрова синтезація (DDS)</b>            | осцилятор із 64-бітним фазовим акумулятором; точна фаза за $\text{mod-}2^{64}$ , фазонеперервний при зміні частоти                                                                             | \$1.1           |
| <b>Сума кутів + синус за Тейлором</b>            | таблиця на 4096 записів для $\theta_0$ + реконструкція <code>sin/cos(<math>\Delta\theta</math>)</code> рядом Тейлора 5-го порядку ( <code>sin(<math>\theta_0+\Delta\theta</math>)=...</code> ) | \$1.2           |
| <b>Експоненційний синусний свіп Фаріни (ESS)</b> | лог-чірп <code>sin(<math>K(e^{t/L}-1)</math>)</code> ; повторно використовується байт-у-байт як еталон для деконволюції                                                                        | \$1.7,<br>\$4.1 |
| <b>Рожевий шум Восс-Маккартні</b>                | генератор $1/f$ сумуванням октав, $O(1)/\text{відлік}$ , $-3$ дБ/окт                                                                                                                           | \$1.9           |
| <b>TPDF-дизер</b>                                | дизер із трикутним розподілом (різниця двох рівномірних) перед квантуванням, декорелює похибку LSB                                                                                             | \$1.13          |
| <b>Спад Ганна / припіднята косинусоїда</b>       | згладжування країв свіпу та придушення витоку у вікні                                                                                                                                          | \$1.8,<br>\$4.1 |

### Спектральні / FFT

| Алгоритм                                                      | Що це таке                                                                                                                                                                        | Де                        |
|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| <b>FFT за Кулі-Тьюкі (основа 2, лема Даніельсона-Ланцоша)</b> | внутрішнє FFT із проріджуванням за часом; лема Д-Л — це рекурсивне розбиття на парні/непарні, яке вона реалізує                                                                   | \$3.1                     |
| <b>Перестановка з реверсом бітів</b>                          | переупорядкування індексів реверсом перед стадіями «метеликів»                                                                                                                    | \$3.1                     |
| <b>Однобінний DFT за Герцелем</b>                             | $O(N)$ магнітуда/фаза на одній частоті без повного FFT — рушій вимірювання <b>точної основної частоти</b> (суб-бінно) і в Осцилографі, і у FFT; повне FFT дає лише найближчий бін | \$2.2,<br>\$2.5,<br>\$3.5 |

|                                                               |                                                                                                                 |                             |
|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|-----------------------------|
| <b>Блекмен-Гарріс (4-членний)</b>                             | вікно аналізу з низькими бічними пелюстками                                                                     | \$3.2                       |
| <b>Наттолл (7-членний Блекмен-Гарріс)</b>                     | вікно з дуже низькими бічними пелюстками (BH7)                                                                  | \$3.2                       |
| <b>Flat-top (SR785, 5-членний)</b>                            | вікно з точною амплітудою                                                                                       | \$3.2                       |
| <b>HFT144D / HFT248D (Heinzel, high-flattop)</b>              | вікна з дуже малим витоком та плоскою амплітудою для вимірювання тонів у великому динамічному діапазоні         | \$3.2                       |
| <b>Вікно Кайзера (KB24 / KB38)</b>                            | вікно на основі $I_0$ при $\beta=24$ / $\beta=38$ — бічні пелюстки $\approx -226$ дБ / нижче подвійної точності | \$3.2                       |
| <b>Вікно Долфа-Чебишова</b>                                   | рівнопульсаційне вікно при 150-300 дБ, побудоване через обернене DFT від $T_{N-1}$                              | \$3.4                       |
| <b>Поліном Чебишова <math>T_n</math></b>                      | тригонометрична/гіперболічна форма, керує вікном Долфа-Чебишова                                                 | \$3.2, \$3.4                |
| <b>Параболічна / квадратична інтерполяція піка</b>            | 3-точкове суб-бінне уточнення піка/частоти                                                                      | \$2.2, \$3.5, \$3.11, \$4.3 |
| <b>Оцінювач за різницею фаз (Квінн /Якобсен)</b>              | суб-бінна частота з міжкадрового приросту фази                                                                  | \$3.5                       |
| <b>Інтерпольований пік за Смітом і Серроу</b>                 | квадратична оцінка значення піка для рівнів IMD-тонів                                                           | \$3.11                      |
| <b>Когерентне (векторне) усереднення + полобова деротація</b> | фазово-узгоджена комплексна сума; стала фаза на пелюстку (а не побінний нахил)                                  | \$3.6, \$3.14               |
| <b>Інтермодуляційні продукти CCIF/DIN</b>                     | сітка різницевих IMD-продуктів та їх відношення                                                                 | \$3.11                      |

## Фільтрація та оцінювання

| Алгоритм                                      | Що це таке                                                                               | Де            |
|-----------------------------------------------|------------------------------------------------------------------------------------------|---------------|
| <b>Інтерполяція Ланцоша (вікнований sinc)</b> | реконструкція Віттакера-Шеннона, $A=16$ пелюсток, ядро з антиаліасинговим масштабуванням | \$2.4         |
| <b>Тригер Шмітта (гістерезис)</b>             | дворівнева кваліфікація фронту для синхронізації осцилографа                             | \$2.1         |
| <b>IIR-ФНЧ Чебишова I роду</b>                | каскад біквадів, ФВЧ-фільтр 80 кГц (увага: застарілий коментар «Butterworth»)            | \$2.5         |
| <b>Білінійне перетворення + передкорекція</b> | відображення полюсів аналог→цифра $\tan(\pi \cdot f_c/f_s)$                              | \$2.5         |
| <b>Медіанний фільтр (де-спайк)</b>            | ковзна медіана, видалення імпульсів зі збереженням фронтів                               | \$2.5         |
| <b>IIR-гребінка зі зворотним зв'язком</b>     | гребінчастий режектор гармонік мережі $(1-z^{-N})/(1-\alpha \cdot z^{-N})$               | \$2.5, \$3.15 |
| <b>Синхронне I/Q (lock-in) детектування</b>   | квадратурна кореляція для відновлення фази биття двох тонів                              | \$2.6         |
|                                               |                                                                                          |               |

|                                                            |                                                                               |       |
|------------------------------------------------------------|-------------------------------------------------------------------------------|-------|
| <b>Деконволюція в частотній області <math>H=Y/X</math></b> | повна передавальна функція (магнітуда+фаза) з однієї пари FFT                 | \$4.3 |
| <b>Поворот лінійної фази (затримка)</b>                    | усунення транспортної затримки DAC↔ADC через $e^{j2\pi k b/M}$                | \$4.3 |
| <b>Часове стробування імпульсної характеристики</b>        | IR із вікном Ганна — компроміс роздільності проти пульсацій (типово вимкнено) | \$4.3 |
| <b>RIAA-корекція (фоно)</b>                                | мережа сталих часу 3180/318/75 мкс (+IEC 7950 мкс)                            | \$4.7 |

## Статистика, згладжування та графіка

| Алгоритм                                  | Що це таке                                                                                        | Де                     |
|-------------------------------------------|---------------------------------------------------------------------------------------------------|------------------------|
| <b>Онлайнова дисперсія за Велфордом</b>   | однопрохідні середнє/ $\sigma$ для живої статистики вимірювань                                    | \$2.3                  |
| <b>Робастний поріг median + MAD</b>       | $\text{median} + k \cdot \text{MAD}$ для рівня шуму / відкидання викидів                          | \$3.10, \$3.12, \$3.13 |
| <b>Згладжування за Савицьким-Голеєм</b>   | ковзний поліноміальний фільтр найменших квадратів (зберігає піки)                                 | \$4.4                  |
| <b>Обернення за Гаусом-Жорданом</b>       | розв'язує коефіцієнти Вандермонда для Савицького-Голея                                            | \$4.4                  |
| <b>Експоненційне ковзне середнє (ЕМА)</b> | згладжує RMS живого вимірювача / детекції фіксації мережі                                         | \$2.10, \$4.9          |
| <b>Відсікання ліній за Лянгом-Барскі</b>  | посегментне відсікання полілінії, щоб далекі координати не «загорталися» ( $\text{GDI} \pm 32k$ ) | \$3.16                 |
| <b>«Гарні числа» 1-2-5 за декадами</b>    | генерація основних/проміжних поділок лог/лінійної осі                                             | \$2.8, \$3.16          |
| <b>Децимація стовпців min / max</b>       | стовпці обвідної на піксель для щільних осцилограм /спектрів                                      | \$2.9, \$3.16          |

## Угоди та спільні примітиви

Кілька фактів повторюються наскрізно; їх викладено тут один раз:

- **Нормалізована область відліків.** Аудіо-відліки скрізь усередині живуть у числах із рухомою комою в діапазоні  $[-1, +1]$ . Перетворення в/із цілих PCM відбувається лише на межі пристрою/файлу.
- **Масштабування напруги.** Нормалізований відлік перетворюється у вольти за  $\text{peakVolts} = \text{adcFsVoltageRms} \cdot \sqrt{2}$  (повна шкала ADC), а вихід генератора масштабується відносно каліброваної повношкальної RMS-напруги DAC ( $\text{dacFsVoltageRms}$  — налаштування, що задається діалогом «Calibrate-DAC» та ін'єктується у конструктор `SignalGenerator`). Це гарантує, що захоплений файл і жива осцилограма показують однакову амплітуду.
- **Однобінний DFT за Герцелем** ( $\text{coeff} = 2 \cdot \cos(2\pi f/fs)$ , магнітуда  $\sqrt{(q_1^2 + q_2^2 - \text{coeff} \cdot q_1 \cdot q_2)}$ ) — робоча конячка для вимірювання магнітуди однієї частоти без повного FFT; з'являється у частотомірі осцилографа, трекері мережевої гребінки та к-перепідгонці FFT.
- **Реконструкція Ланцоша (вікнований sinc)** використовується **лише** у часовому поданні осцилографа (`ScopeLanczos`); шлях частотної характеристики ресемплює свої збережені криві лог-частотною інтерполяцією, а не Ланцошем.

- **Спільне побудування графіків у частотній області.** Подання FFT та Частотної характеристики поділяють один графічний рушій ( `AbstractMeasurementView` / `AbstractFreqDomainView` ) для «гарних» поділок лог/лінійних осей, відображення значення→піксель та рендерингу відсиченої полілінії. Описано один раз у §3.16.

## Зміст

---

1. **Генератор сигналів** — синтез DDS, математика форм, свіпи, прив'язка до FFT-бінів, дизер/PCM, вихідні бекенди
  2. **Осцилограф** — синхронізація, вимірювання частоти сирого сигналу, реконструкція Ланцоша, DSP-фільтри, відновлення биття, рендеринг
  3. **FFT-аналізатор** — FFT основи 2, вікна, когерентне усереднення та фазова фіксація, THD /IMD/рівень шуму, розтягнення пелюстки, відкидання розривів, придушення мережі, спільне побудування
  4. **Частотна характеристика** — деконволюція лог-свіпу Фаріні, згладжування Савицького-Голея, калібрування `.frc`, RIAA-корекція, порівняння/різниця, живе вимірювання
- 

## 1. Генератор сигналів

---

Генератор сигналів синтезує тестові форми сигналу відлік за відліком у числах із рухомою комою, пакує їх у PCM і передає до DAC через один із трьох аудіо-бекендів. Ядро — `SignalGenerator.java`; обв'язка GUI у `GeneratorController/GeneratorPane`; кодування PCM та ввід/вивід пристрою — у класах `*Generator` для кожного бекенда та у файлових експортерах.

### 1.1 Пряма цифрова синтезація (осцилятор із фазовим акумулятором)

Усі періодичні форми керуються **64-бітним фазовим акумулятором**, що зберігається у `long` і просувається на фіксований `phaseInc` щовідлік ( `SignalGenerator.java`, `nextSample()` ).

- **Приріст фази:** `phaseInc = round(f / fs · 264)` ( `toPhaseInc`, використовується конструкторами і `setFrequency` ). Повний оберт дорівнює 2<sup>64</sup>, тож звичайне переповнення `long` і є завертанням фази за модулем 2<sup>64</sup> — без маскуванню, без посеместного дрейфу. Приріст обчислюється у `double`, чия 53-бітна мантиса обмежує корисну роздільність до `fs/253 ≈ 43 пГц` при 384 кГц — сенс цієї ширини у запасі для контуру фіксації частоти: суб-ppb-корекції лишаються представними, а не квантуються в нуль. `phaseInc` є `volatile`, тож зміни частоти з GUI-потоків доходять до аудіо-потоків атомарно та **фазонеперервно** — акумулятор зберігається при зміні частоти, тож кліку немає.
- **Живі заміни форми/частоти** зберігають акумулятор, тож переходи між синус/трикутник/прямокутник плавні ( `setForm` ).
- **Другий акумулятор** `phaseAcc2/phaseInc2` існує винятково для `DUAL_TONE` і просувається лише коли ця форма активна.

### 1.2 Синтез синуса — таблична вибірка + корекція за Тейлором

`ddsSine()` читає **4096-записну** ( `TABLE_BITS=12` ) повнохвильову `SINE_TABLE/COS_TABLE`, побудовану при завантаженні класу. Старші 12 бітів акумулятора обирають грубий кут  $\theta_0$ ; решта 52 молодші біти формують дробову похибку фази  $\Delta\theta$  (радіани, через `TWO_PI_OVER_2_64` ). Точний відлік відновлюється тотожністю суми кутів:

---

$$\sin(\theta_0 + \Delta\theta) = \sin(\theta_0) \cdot \cos(\Delta\theta) + \cos(\theta_0) \cdot \sin(\Delta\theta)$$

де  $\cos(\Delta\theta)$  і  $\sin(\Delta\theta)$  — **ряди Тейлора 5-го порядку**, обчислені за схемою Горнера ( $\cos \approx 1 - \Delta\theta^2/2! + \Delta\theta^4/4!$ ,  $\sin \approx \Delta\theta - \Delta\theta^3/3! + \Delta\theta^5/5!$ ). Оскільки  $\Delta\theta$  обмежено одним кроком таблиці ( $\approx 1.53$  мрад), похибка зрізання  $\sim \Delta\theta^7/7! \approx 1e-24$  лежить далеко нижче подвійної точності. `ddsSine2()` — байт-ідентична копія, що читає `phaseAcc2`, тримана окремо, щоб JIT інлайнив подвійнотональне читання тонів без копіювання параметра.

### 1.3 Прямокутник, трикутник і пилка (кусково-лінійні, точні)

Вони відображають акумулятор у нормалізовану фазу `frac`  $\in [0,1)$  і точні за побудовою (член Тейлора не потрібен):

- **Прямокутник / імпульс** (`ddsRectangle :810`): `+1` поки `frac < rectDuty`, інакше `-1`. Шпаруватість  $\neq 0.5$  навмисно дає ненульову постійну складову для імпульсних тестових сигналів. Шпаруватість затиснута в `[0.001, 0.999]`, щоб послідовність ніколи не вироджувалась у DC (`setRectangleDuty :519`).
- **Трикутник із регульованою шпаруватістю** (`ddsTriangle :823`): зростаючий пандус `-1→+1` на першій частці `triDuty`, спадний пандус `+1→-1` на решті. `triDuty=0.5` — симетричний трикутник; значення біля 1.0/0.0 наближаються до пилки / оберненої пилки. Затиснуто `[0.001, 0.999]` (`setTriangleDuty :532`).

### 1.4 Подвійний тон (тестовий сигнал IMD)

`DUAL_TONE` додає два незалежні DDS-синуси: `ddsSine()·w1 + ddsSine2()·w2` (`:554`). Лінійні ваги на тон `dualW1`, `dualW2` кожна дорівнює `amplitude_percent/100`, затиснуті в `[0,1]` незалежно (`setDualToneAmplitudes :494`). Два некорельовані синуси дають сукупний  $RMS = \sqrt{(w_1^2 + w_2^2)/2}$  (`rawRmsDualTone :426`), а загальний масштаб `amplitude` **перенормовується відносно цього RMS** щоразу, коли змінюється розподіл або `Vrms`, тож сукупний вихідний `Vrms` завжди дорівнює полю `Amplitude` незалежно від розподілу (50/50 не тихіше за 100/0). Дільник обмежено знизу `1e-12`, щоб уникнути NaN, коли обидва тони 0 %. Тон 2 прив'язується до FFT-біна незалежно від тону 1 (див. 1.10).

### 1.5 Гармонічно-компенсований синус (передспотворення)

`SINE_COMPENSATED` (`ddsSineCompensated`) віднімає виміряні гармоніки спотворення від чистого DDS-синуса, щоб залишкові  $H_2..H_n$  аналогового тракту скомпенсувались. Фаза кожної гармоніки генерується **цілочисельним множенням головного акумулятора** на номер гармоніки `h`: `hPhase = phaseAcc·hNum + hPhaseOff64` (завертається за  $\text{mod } 2^{64}$ ; початкові фази кожної гармоніки попередньо переводяться у 64-бітні фазові зсуви при першому використанні). Це жорстко фіксує корекційний тон точно на `h·f_DDS` — без дрейфу чи биття проти реального спотворення — а корекція `s -= hAmpRatio[i]·ddsCos(hPhase)` працює на тому ж ядрі таблиця+Тейлор, що й основа (`ddsCos`), тож жоден `Math.cos` не сидить на посеместному шляху ( $\sim 2$  млн тригонометричних обчислень/с при 5 гармоніках  $\times$  384 кГц).

**Фазове калібрування системної затримки** (`loadHarmonics :925`, `loadHarmonicsFromResult :864`): DDS-синус при `n=0` має FFT-фазу  $-\pi/2$ , тож виміряна фаза основи дає затримку контуру  $\omega D = -(\varphi_1 + \pi/2)$ . Це переводиться у рад/Гц (`delayRadPerHz = \omega D / f_fund`), і фаза випромінювання кожної гармоніки — `\varphi_init = \varphi_h_measured + f_h·delayRadPerHz`, тож корекція приходить до ADC з фазою, що обнуляє виміряну гармоніку. Амплітуди зберігаються як відношення до основи (`amplitude_pct/100`), тож вони незалежні від рівня. Корекції завантажуються з `fft_harmonics_*.csv`, `applied_compensation_*.csv`, безпосередньо з `FftResult`

, або з попередньо накопичених комплексних корекцій (ітеративний робочий процес). Некогерентні FFT-результати не несуть фази, тож фазова корекція вимикається з попередженням і застосовується лише амплітудна корекція ( :865 ). Фаза CSV береться зі стовпців `re/im` із повною точністю, якщо вони є, інакше з `phase_deg`. Якщо калібрувальна амплітуда CSV відрізняється від запитаного `Vrms`, погармонічні відношення перемасштабовуються на `csv_amplitude/amplitudeVrms`.

## 1.6 Лінійний (сталовидкисний) частотний свіп / чірп

`LINEAR_SWEEP` ( `linearSweepNext` :574 ) — фазонеперервний чірп, чия **миттєва частота лінійно зростає**  $f_0 \rightarrow f_1$  упродовж `sweepPeriodSamples`:

```
instF = f0 + (f1-f0)·(sweepIdx/period)
s      = sin(sweepPhase);  sweepPhase += 2π·instF/fs
```

Фаза накопичується у `double` і завертається за модулем  $2\pi$ , щоб обмежити магнітуду ( :585 ). Вона рестартує фазонеперервно щотиж, коли `sweepLoop` істинне, інакше замовляє після одного циклу. Може застосовуватись **спад Ганна** на цикл (див. 1.8).

## 1.7 Логарифмічний (Фаріни) експоненційний синусний свіп + еталон для деконволюції

`LOG_SWEEP` — експоненційний чірп Фаріни, **попередньо відрендерений** при одиничній амплітуді у `logSweepBuffer` ( `renderLogSweep` :367 ):

$$x(t) = \sin(K \cdot (\exp(t/L) - 1)), \quad L = T/\ln(f_1/f_0), \quad K = 2\pi \cdot f_0 \cdot L$$

Миттєва частота просувається лог-лінійно  $f_0@t=0 \rightarrow f_1@t=T$ ; відлік 0 точно нуль, тож він конкатенується з провідною тишею без розриву. Буфер надається через `getLogSweepBuffer()`, тож аналізатор частотної характеристики повторно використовує **байт-ідентичний еталон X(t)** для FFT-деконволюції  $Y(f)/X(f) \rightarrow H(f)$ . Відтворення ( `logSweepNext` :600 ) видає `logSweepLeadIn` нулів, відтворює буфер, а при зацикленні завертається на початок буфера (минаючи провідну частину). Живі зміни старту/стопу/тривалості перерендерюють буфер ( `rebuildLogSweepBuffer` :697, `setSweepDurationSamples` :676 ).

## 1.8 Спад Ганна на вході/виході

`sweepEnvelope()` ( :625 ) застосовує спад припіднятого косинуса (Ганна) на краях кожного циклу свіпу: вхідний спад  $0.5 \cdot (1 - \cos(\pi \cdot \text{idx}/\text{fadeIn}))$  на перших `fadeInSamples`, дзеркальний вихідний спад на останніх `fadeOutSamples`, і 1.0 у сталому середньому. Довжини спадів затискаються до половини циклу, щоб вхідний і вихідний спади не перекривались ( `setSweepParams` :642 ).

## 1.9 Генератори шуму (білий, рожевий Восса-Маккартні)

- **Білий шум** ( :548 ): `rng.nextGaussian()` — гаусів,  $\sigma=1$ .
- **Рожевий шум** ( `pinkNoise` :843 ): алгоритм **Восса-Маккартні** із сумуванням октав, `PINK_OCTAVES=16` рядів плюс один завжди-оновлюваний білий член. На кожному такті ряд, індексований кількістю кінцевих нульових бітів лічильника, оновлюється і підтримується поточна сума, даючи  $O(1)$  на відлік спектр  $\approx -3$  дБ/октава ( $1/f$ ). Два варіанти: **гаусове** джерело ( `PINK_NOISE` ) та **рівномірне / плоскоамплітудне** джерело `rng.nextDouble()·2-1` ( `PINK_NOISE_LINEAR` ).

## 1.10 RMS-нормалізація (В RMS → лінійна пікова амплітуда)

Цільовий вихід у **вольтах RMS** переводиться у внутрішній лінійний піковий масштаб за `amplitude = Vrms / (dacFsVoltageRms * rawRms(form))` (конструктори, `setAmplitudeVrms`). `rawRms()` повертає аналітичний RMS одиничної амплітуди кожної форми: синус/свіп  $1/\sqrt{2}$ , трикутник  $1/\sqrt{3}$  (незалежно від шпаруватості), прямокутник 1 (за будь-якої шпаруватості), білий шум 1, рожевий  $1/\sqrt{N+1}$  (гаусів) або  $1/\sqrt{3(N+1)}$  (рівномірний), подвійний тон  $\sqrt{((w_1^2 + w_2^2)/2)}$ . `dacFsVoltageRms` — це **калібрована повношкальна RMS-напруга DAC** — налаштування, що задається діалогом «Calibrate-DAC», ін'єктоване у конструктор генератора (саме воно ніколи не лізе у Preferences) та проштовхнуто наживо через `setDacFsVoltageRms` при перекалібруванні, тож запущений тон перемасштабовується без рестарту. Оскільки відношення пік/RMS різне для кожної форми, перемикання форм переділяє кешований `currentVrms`, щоб вихідний `Vrms` лишався сталим (`setForm`).

## 1.11 Прив'язка до FFT-біна (округлення до когерентної частоти)

`FftBinSnap.snapIfEnabled()` округлює запитану частоту до найближчого центра FFT-біна, щоб тон потрапив точно на бін (уникає спектрального витоку у когерентному FFT-аналізі): `binHz = fs/fftLength; snapped = round(raw/binHz) * binHz`. Спрацьовує лише для SINE та DUAL\_TONE, коли користувач увімкнув прив'язку-до-біна; усі інші форми проходять без змін. Кожен тон подвійного тону прив'язується незалежно. Прив'язане значення — це те, що DDS фактично випромінює, тоді як *сире* введенне значення зберігається (`GeneratorController.java: 98, :183`). Подання FFT перепублікує нову довжину FFT, тож запущений тон переприв'язується наживо, а контур фіксації частоти публікує суб-Гц-корекції (`GENERATOR_FREQ_TRIM`), що наживо застосовуються до DDS.

## 1.12 Скидання стану світу після JIT-прогріву

`resetSweepPosition()` (:388) обнуляє `logSweepIdx`, `sweepIdx`, `sweepPhase` та `phaseAcc`. Кожен бекенд викликає це **після** перед-стрімового JIT-прогріву, який спожив виклики `nextSample()` — інакше вимірювання частотної характеристики захопило б світ, що почався посеред буфера, тоді як еталон деконволюції починається з нуля, ламаючи вирівнювання.

## 1.13 Перетворення float → PCM: TPDF-дизер + квантування

Пакування PCM централізоване у `PcmQuantizer` (`sound/PcmQuantizer.java`), яким володіє кожен бекенд відтворення (`JavaSoundGenerator`, `WasapiGenerator`, `WdmksGenerator`), тож стрімовані байти ідентичні незалежно від шляху; файлові експортери використовують ту ж математику дизеру/квантування:

- **TPDF-дизер:** `tpdfNoise = (rng.nextDouble() - rng.nextDouble()) / 2^(ditherBits-1)`. Різниця двох рівномірних випадкових — трикутний розподіл, що охоплює  $\pm 1$  LSB при обраній бітності; доданий перед квантуванням, він декорелює похибку квантування. `ditherBits=0` вимикає його; кількість живо-налаштовна (`volatile`, читається на відлік) і обмежена бітністю виходу. RNG — `SplittableRandom`, обмежений потоком рендеру (без посеместної вартості монітора/CAS).
- **Затиск + квантування:** відлік затискається в  $[-1, 1]$ , далі `pcm = round(sample * (2^(N-1)-1))` записується **little-endian**; 8-бітне є **знаковим** (`round(s * 127)`) — усі бекенди відкривають свої лінії/стріми у знакових форматах.
- **Перемежування каналів:** стерео, **обидва канали несуть однаковий відлік** (моносигнал продубльований L/R).

- Шар синтезу лишається без дизеру, тож експорти та еталон деконволюції  $X(t)$  лишаються ідеальною формою сигналу; дизер існує лише на межі пристрою, де його амплітуда  $\pm 1$  LSB визначається цільовою бітністю.

## 1.14 Експорт у файл (WAV / AIFF / FLAC)

`SignalFileExporter` обирає контейнер за розширенням (`.wav` / `.flac` / `.aif` / `.aiff`) і пише через `WavWriter` / `AiffWriter` / `FlacWriter` за спільним `PcmSink`, використовуючи той самий шлях дизеру/квантування `fillBuffer`, що й живе відтворення. Для **періодичних** форм він зрізає довжину до найбільшого цілого кратного одного періоду сигналу (`truncateToFullPeriods :95`, `period = round(fs/freq)`), щоб файл зациклювався без клацання; шум/свіпи використовують запитану тривалість як є. `WavSignalExporter` — лише-WAV застарілий варіант. 8-бітний експорт лише для WAV (FLAC відхиляє 8-бітне).

## 1.15 Механіка виходу для кожного бекенда

Три бекенди реалізують `AudioPlayback`; GUI/CLI керують ними однаково через `AudioBackend`. Усі роблять **~500 мс JIT-прогрів** гарячого шляху кодування перед тим, як пристрій потягне реальне аудіо (щоб C2 скомпілював `nextSample/encode` і деоптимізації втеглися), потім `resetSweepPosition()`.

- **JavaSound** (`JavaSoundGenerator`): блокувальний `SourceDataLine.write()` зі стрімінгом по 4096-кадрових порціях; передзаповнює апаратний буфер перед сигналом готовності. Планування реального часу віддане нативному мікшеру (нуль JNA-обходів), що коментарі звуть шляхом без розривів. (Це бажаний шлях: прямий цикл рендеру WASAPI періодично вставляє однопіріодний плоский провал, бо робить п'ять JNA-обходів на такт без реєстрації MMCSS — маршрутизація DDS через `SourceDataLine` JavaSound уникає цього.)
- **WASAPI** (`WasapiGenerator`): ексклюзивний режим **подієвого зворотного виклику** (зі спільним резервним), що тягне через `IAudioRenderClient::GetBuffer/ReleaseBuffer` на кожне пробудження дескриптора події. Передзаповнює весь HW-буфер перед Start (інакше ексклюзивний режим дає збій на першому такті). Наглядач циклу рендеру відстежує інтервал такту проти порогу недозаповнення `expectedTickNanos·1.2` і бюджету заповнення  $\frac{1}{2} \cdot \text{такт}$ , логуючи пізні такти / повільні заповнення / часткові заповнення кожні 5 с. Кодує у перекузовний нативний скретч-буфер (без алокацій на гарячому шляху). Вважається застарілим шляхом (див. примітку про JavaSound вище).
- **WDM-KS** (`WdmksGenerator`): режим **зворотного виклику** PortAudio WDM-KS — `paCallback` заповнює вихідний вказівник на потоці реального часу PortAudio, повертаючи `paContinue` / `paComplete` залежно від стоп-прапора та лічильника кадрів, і рахує прапори `paOutputUnderflow` / `paOutputOverflow` (зливаються у обмежені за частотою попередження). Використовує `defaultHighOutputLatency` пристрою та `paFramesPerBufferUnspecified`, тож KS-пін обирає природний розмір (нульова латентність чи нав'язаний розмір буфера підвищує WDM-KS після праймінг-виклику).

Потік відтворення генератора працює на `Thread.MAX_PRIORITY`, щоб тримати ексклюзивні бекенди попереду GC/GUI і уникати провалів посеред плато (`GeneratorController.java:220`).

## 1.16 Відтворення файлу («Load from...»)

`FilePlayController` декодує обраний WAV/AIFF/FLAC (FLAC через `jflac` SPI) через `javax.sound.sampled` і стрімить декодований PCM у стандартний `SourceDataLine`, перевіряючи стрім із файлу на EOF при зацикленні. Він навмисно **не** маршрутизований через `AudioBackend` — це зручність моніторингу, а не вихід вимірювальної якості.



## 1.17 Числові / одиничні алгоритми GUI

- **Одиниці амплітуди** (`AmplitudeUnit`): усі перетворення канонікалізуються через  $V_{\text{RMS}}$ .  $\text{dBV } V_{\text{rms}} = 10^{(\text{dBV}/20)}$ ;  $\text{dBFS } V_{\text{rms}} = f_s \cdot 10^{(\text{dBFS}/20)}$  за повношкальним RMS ADC; обернене використовує  $20 \cdot \log_{10}(\cdot)$  обмежене знизу  $1e-12$ , щоб уникнути  $-\infty$ . мкВ/мВ /В — чисто метричні масштабування з групою авто-перемасштабування.
- **Покрокова шпаруватість за відліками** (`stepDutyBySamples` `GeneratorPane.java:1271`): крок стрілки/колеса шпаруватості зсуває верхній/нижній фронт точно на **один відлік поточного періоду** ( $n = \text{round}(f_s/f_{\text{req}})$ ), переводячи шпаруватість%  $\leftrightarrow$  лічильник відліків  $k$  і затискаючи  $k \in [0.01n], [0.99n]$ , тож сітка шпаруватості відстежує цілочисельну роздільність форми сигналу.
- **Крокування частоти**: колесо =  $\pm 5\%$  мультиплікативно, стрілки =  $\pm 1$  Гц адитивно, обмежено знизу 0.01 Гц.

## 1.18 Піктограми форм сигналу (векторне малювання SWT)

`SignalFormIcon` рендерить 24×24 прозоро-ARGB піктограму на форму з антиаліасованими SWT-примітивами `Path/drawLine`, кешовану на `Display`. Помітна математика: гліф подвійного тону малює  $\sin(5\pi t) + \sin(6\pi t)$ , щоб показати обвідну биття; гліфи свіпу використовують фазу  $\propto t^2$  (лінійний) проти  $\propto \exp(3t) - 1$  (логарифмічний), щоб візуально стиснути період зліва→направо; гліфи шуму використовують засіяний `Random` (детермінований на варіант) із 2-/3-прохідним усередненням сусідів, щоб відрізнити білий від рожевого.

## 2. Осцилограф

Модуль осцилографа захоплює аудіо звукової карти у кільцевий буфер, рендерить синхронізовану осцилограму з суб-відлічним прив'язуванням і обчислює живі вимірювання часу/амплітуди на фоновому робітнику. Рендеринг працює на частоті перемалювання SWT (~60+ Гц); вимірювання — на окремому робочому потоці 10 Гц. Наскрізно нормалізовані відліки живуть у  $[-1, +1]$  і масштабуються у вольти за  $\text{peakVolts} = \text{adcFsVoltageRms} \cdot \sqrt{2}$  (напр., `ScopeView.java:2360`).

### 2.1 Синхронізація (детекція фронту, гістерезис, суб-відлічне уточнення)

**Тригер Шмітта з гістерезисною смугою** — `ScopeTrigger.find` (`ScopeTrigger.java:70`). Обходить `data[from..to)` із двома порогами  $lo = \text{level} - \text{hysteresis}$ ,  $hi = \text{level} + \text{hysteresis}$ . Зростаючий тригер *кваліфікується* лише коли сигнал, попередньо опустившись нижче  $lo$  (стан Шмітта  $-1$ ), фіксує перетин  $level$  і потім піднімається чисто вище  $hi$ ; симетричне правило діє для спадних фронтів. Вхідний стан Шмітта засівається скануванням назад від `from` до першого відліку, надійно за межами мертвої зони (`ScopeTrigger.java:79`). При  $\text{hysteresis} == 0$  два пороги колапсують на  $level$ , відновлюючи звичайну односеместну скобкову синхронізацію. Функція повертає дробовий індекс **найправішого** (найсвіжішого) кваліфікованого перетину, або  $-1$ . Гістерезис виражається в екранних одиницях:  $\text{hysteresisDiv} \cdot \text{vDiv} / \text{peakVolts}$  переводить налаштування «поділок екрана» у нормалізовані відлічні одиниці (`ScopeView.java:1956`).

**Holdoff / мінімальний інтервал** — друге перевантаження `find` (`ScopeTrigger.java:70`) відхиляє кваліфікований перетин ближчий за `minSpacingSamples` до останнього прийнятого.

Використовується у режимі подвійного тону, щоб тримати один тригер на биття `|F1-F2|` (`minSpacingSamples = sampleRate / |F1-F2|`).

**Суб-відлічне уточнення перетину** — два методи:

- *Лінійний* (`ScopeTrigger.linear`, `ScopeTrigger.java:135`): `prevIdx + (level - prev)/(curr - prev)`.
- *Sinc-бісекція* (`ScopeTrigger.refine`, `ScopeTrigger.java:148`): 10-ітераційна бісекція смугово-обмеженої реконструкції `ScopeLanczos.lanczos(data, n, m, 1.0)` між скобуючими відліками, даючи точність  $2^{-10} \approx 0.001$ -відліку. Власне sinc-інтерполяційне налаштування тригерного каналу обирає, який використовується (`ScopeView.java:1922`).

**Обчислення рівня тригера** — повзунок рівня — це частка висоти полотна; вона відображається у нормалізоване значення відліку через `(triggerCenterY - triggerLevelY) / vScaleTrig`, де `vScaleTrig = peakVolts/vDiv * pixelsPerDivY` (`ScopeView.java:1931`). У режимі AC-зв'язку показана осцилограма має знятий DC-зсув, тож усереднене DC-середнє каналу додається назад у поріг, щоб компаратор спрацьовував там, де користувач бачить пунктирну лінію рівня (`ScopeView.java:1951`).

**Режими** (`TriggerMode: AUTO / NORMAL / SINGLE`, `TriggerMode.java`; `TriggerEdge: RISE / FALL`, `TriggerEdge.java`). Тристадійна логіка прив'язування (`ScopeView.java:2017 - 2088`): (1) якщо знайдено свіжий тригер, прив'язати вікно так, щоб центральний піксель відображався точно на `triggerFrac`, розщеплюючи дробовий лівий край вікна на ціле (`dispStart`) + дріб (`subSampleOffset`), щоб тримати непарнодовгі вікна центрованими без зсуву на 0.5 відліку; (2) інакше тримати попередній тригер, якщо його абсолютна позиція ще в буфері; (3) інакше AUTO/SINGLE вільно біжать по найсвіжіших відліках, тоді як NORMAL лишає панель пустою. Позиція тригера зберігається як *абсолютний* `writePos (lastTriggerAbsPos)`, тож той самий час лишається центрованим між перемалюваннями, поки кільце прокручується (`ScopeView.java:2033`). SINGLE заморожує кадр у виділені буфери `capturedLeft/Right (captureSingleFrame, ScopeView.java:2307)`, незалежні від кільця, тож вони переживають невизначено довго.

**Розмірення pre/post-roll** — вікно читання — `2*displaySamples + 2*LANCZOS_PADDING + extraLookback` (`ScopeView.java:1852`), гарантуючи повне вікно показу обох боків будь-якого налаштування повзунка позиції тригера плюс додаткове озирання (до `MEAS_MAX_SAMPLES`), щоб низькочастотні сигнали все ще містили фронт. Діапазон пошуку тригера затискається асиметрично за `triggerPosFrac`, щоб результуюче вікно показу лишалось у буфері (`ScopeView.java:1914`).

## 2.2 Вимірювання основної частоти (сирий сигнал, засіяний гребінкою)

Частота вимірюється у `SignalMeasurements.compute` (`SignalMeasurements.java:72`) стратегією **засів перетинами** → **уточнення Герцелем**, навмисно по *сирому* (не-гребінчасто-фільтрованому) сигналу:

1. **Грубий період за пороговими перетинами** — рахуються зростаючі перетини напівамплітудної середини `(min+max)/2` (`SignalMeasurements.java:102`); два найкрайніші перетини охоплюють ціле число циклів, даючи `coarsePeriod = (lastCross - firstCross)/(crossCount - 1)` (`SignalMeasurements.java:166`). Періоди з перетинів відстежують справжню основу напряду, тож вузькошпаруватий прямокутник дає правильну відповідь, а не замикається на майже-рівноамплітудній гармоніці (за коментарем на : 160).

2. **Тристадійне уточнення Герцелем** — `refineAroundEstimate` (`SignalMeasurements.java:229`): грубе сканування (100 проб по півширині), потім два послідовні сканування, кожне зводить крок у 20х, потім **параболічна інтерполяція** по трьох суміжних точних бінах ( :264 ) для суб-крокової точності ( $\leq \sim 0.005$  Гц на 1-с буфері), при фіксованих  $\approx 180$  обчисленнях Герцеля.
3. **Контроль якості** — відношення фіксації `mag/n/rmsAc` ( $\approx 0.707$  для чистого синуса,  $\sim 0.005$  для білого шуму) має перевищити 0.1, щоб допустити прямокутники аж до  $\sim 1\%$  шпаруватості (`SignalMeasurements.java:193`).
4. **Широкосмуговий резерв** — коли перетини виглядають ненадійно (відношення  $< 0.1$ , тобто  $\text{SNR} < \sim 10$  дБ), `scanForFundamental` (`SignalMeasurements.java:293`) сканує всю смугу на 0.1-с підвікні з нативною роздільністю FFT-біна, потім уточнює пік.
5. **Кінцева оцінка з компенсацією витоків** — обраний пік перевуточнюється на **вікнований Ганном** копії (`hannWindowed`, `SignalMeasurements.java:334`); низькі бічні пелюстки вікна придушують образ негативної частоти, чий витік тягне пік Герцеля прямокутного вікна донизу на коротких буферах ( $\sim 0.3$  Гц при 20 циклах), приколюючи частоту до  $< 0.02$  Гц (`SignalMeasurements.java:198`).

**Магнітуда однобінного DFT за Герцелем** — `goertzelMagnitude` (`SignalMeasurements.java:314`): `coeff = 2cos(2 $\pi$ f/fs)`, рекурентність `q0 = (x-mean) + coeff·q1 - q2`, магнітуда  $\sqrt{(q1^2 + q2^2 - \text{coeff} \cdot q1 \cdot q2)}$ . DC віднімається, тож внесок дає лише AC-складова на `f`.  $O(N)$  на пробу — FFT не потрібне.

**Чому сирий, а не вихід гребінки** — коли мережева гребінка каналу увімкнена, робітник (`ScopeMeasurementWorker.computeMeasurementOnce`, `ScopeMeasurementWorker.java:392 - 447`) використовує гребінку лише щоб зробити тон спектральним *піком* (надійний засів), бо її режекторні провали стоять на кожній гармоніці мережі і один може потрапити за кілька Гц від тону (а на високих частотах дискретизації гребінка ніколи не вляжеться всередині вікна) — обидва тягнуть гребінчасту частоту донизу. Він тримає сиру копію обраного каналу (`rawSelBuf`, :404) і перезаколює засів на сирому сигналі через `refineFrequencyAround` (`SignalMeasurements.java:377`) — пошук Герцелем у вікні Ганна по вузькій смузі `seed  $\pm$  FREQ_REFINE_HALF_HZ` (2 Гц, :71), достатньо вузькій, щоб жодна мережева гармоніка ( $\geq \sim 50$  Гц віддалік) не конкурувала.

## 2.3 Вимірювання амплітуди / RMS / середнього / Vpp

У `SignalMeasurements.compute` (`SignalMeasurements.java:80 - 96`): один прохід накопичує `sum`, `sumSq`, `min`, `max`. Далі:

- **Vmean** = `mean · peakVolts` (DC-зсув у вольтах).
- **Vpp** = `(max - min) · peakVolts`.
- **Vrms** = AC RMS —  $\sqrt{(\text{variance}) \cdot \text{peakVolts}}$ , де `variance = E[X2] - E[X]2 = sumSq/n - mean2` (`SignalMeasurements.java:92`). DC-зсув навмисно знятий, тож `Vrms` не вироджується до  $\approx V_{\text{mean}}$  на DC-зміщеному шумовому сигналі.

**Час наростання/спаду** — `riseTime/fallTime` (`SignalMeasurements.java:116 - 153`): скобкові переходи 10%↔90%, пороги `lo = min + 0.1 · (max-min)`, `hi = min + 0.9 · (max-min)`, усереднені по всіх повних переходах із лінійною інтерполяцією для суб-відлічної роздільності.

**Шпаруватість** — частка відліків  $\geq$  напівамплітудного порогу, `highCount/n` (`SignalMeasurements.java:108`, :214).

**Онлайнова віконна статистика** — `MeasurementStats` (`MeasurementStats.java`) накопичує `avg/min/max/σ` методом **Велфорда** (`mean += δ/count`, `m2 += δ·(v-mean)`, `σ = √(m2/(count-1))`), пропускаючи NaN, щоб такти з пропущеним циклом не отруювали вікно. Агрегується по вікну усереднення користувача обходом кільця історії (`prepareMeasurementRows`, `ScopeView.java:953`). `MeasurementRow` (`MeasurementRow.java`) обирає SI-префікс за магнітудою значення, тож `cur/avg/min/max/σ` поділяють одиницю; частота обмежена 7 значущими цифрами (`fmtFreq`, `:71`).

**Подвійний тон** — дві одночасні основи не мають єдиного `Tr/f`/шпаруватості, тож робітник очищає всі часові поля через `withoutTimes` (`SignalMeasurements.java:353`), рендерячи `---`.

## 2.4 Sinc-реконструкція Ланцоша (інтерполяція показу + уточнення тригера)

`ScopeLanczos.lanczos` (`ScopeLanczos.java:82`) реконструює осцилограму на будь-якій дробовій позиції відліку `t` із попередньо обчисленої фазової таблиці — смугово- обмежена (Біттакера-Шеннона) інтерполяція. Ядро: вікнований `sinc` `sinc(x)·sinc(x/A)` із **A = 16 пелюсток** з кожного боку (`LANCZOS_A`,  $\geq 80$  дБ смуга затримування, `:45`). `sinc(x) = sin(πx)/(πx)` з рядом Тейлора 8-го порядку для  $|x| < 0.1$ , щоб уникнути скорочення біля нуля (`ScopeLanczos.sinc`, `:187`).

Параметр `scale` розширює ядро, щоб діяти як **антиаліасинговий ФНЧ на вихідній частоті** (`scale = max(1, samplesPerPx)`), убиваючи енергію між вихідним Найквістом і вхідним Найквістом, що інакше згорнулась би у хибні обвідні биття. Ядра попередньо випікаються у 1024-фазну таблицю (`buildKernelTable`, `:155`), кожен ряд нормований на одиничне підсилення DC ( $\Sigma w = 1$ ), тож похибки підсилення при вузькому V/поділку (помножені на  $\sim 3 \cdot 10^5$  `vScale`) не стискають амплітуду. Двослотовий кеш тримає `scale==1` постійно (використовується уточненням тригера та швидкими часовими рендерами) плюс один інший масштаб, а пошук відділено від синхронізованого шляху промаху, щоб HotSpot C2 інлайнив гарячий цикл (`getKernelTable`, `:124`). Внутрішня сума віднімає `baseline` (центральный відлік) перед зважуванням, тож накопичення відображає *різниці* відліків, а не абсолютні магнітуди — критично при екстремальному V/поділку, де  $\sim 1e-7$  float-епсилон  $\times \sim 3e5$  `vScale` стає видимим піксельним відхиленням (`ScopeLanczos.java:95-109`). `ZoomedView` має власний простіший некешований Ланцош для 1-секундної оглядової смуги (`ZoomedView.java:190`).

## 2.5 DSP-фільтри

**ФНЧ Чебишова I роду** (`LowPassFilter.java`) — фільтр видалення ВЧ-сплесків 80 кГц (`LpfMode.HZ_80`), каскад біквадів 2-го порядку, порядок 8 (`SCOPE_HF_LPF_ORDER`, `ScopeMeasurementWorker.java:65`). І рід із пульсацією смуги пропускання 0.5 дБ (`RIPPLE_DB`, `:53`), обраний над Баттервортом за крутіший перехід. Кожна спряжена аналогова пара полюсів відображається через **білінійне перетворення з передкорекцією частоти** `kWarp = tan(π·fc/fs)` (`LowPassFilter.java:87`); полюси йдуть з `v0 = asinh(1/ε)/N`, `sp = -sinh(v0)·sin(θ)·kWarp`, `op = cosh(v0)·cos(θ)·kWarp` (`:82-93`); секції нормуються на одиничне підсилення DC і працюють у Direct-Form-II transposed (`process`, `:122`). Нижче стробування Найквіста він `inactive` і пропускає (тож 80 кГц — по-ор при 48/96 кГц, працює лише на високих частотах дискретизації). Скидається перед кожним перекривним блоком читання.

Примітка: Javadoc і документація параметра `order` кажуть «Butterworth», але реалізація — Чебишов I роду (коментарі рівня класу та `RIPPLE_DB` правильні) — застарілий коментар.

**Ковзна медіана де-спайк** (`MedianFilter.java`) — `LpfMode.DESPIKE`, вікно 7 (`LpfMode.java:28`). Виводить медіану вікна, центрованого на кожному відліку (`process`, :52; медіана вставкою-сортуванням, :67), видаляючи імпульсні викиди завширшки до `window/2` зі збереженням фронтів без дзвону. Без пам'яті між блоками (без скидання, без крайового перехідного процесу); краї вікна стискаються, а не завертаються.

**Часова мережева гребінка** (`MainsCombFilter.java`) — частотно-відстежувана IIR-гребінка зі зворотним зв'язком  $H(z) = (1 - z^{-N}) / (1 - \alpha \cdot z^{-N})$ ,  $N = \text{sampleRate} / f_0$ , що ставить режекторні провали на DC і кожній гармоніці  $k \cdot f_0$  (:24). Радіус полюса  $\rho = \exp(-\pi \cdot BW / f_s)$ ,  $\alpha = \rho^N$  задає рівномірну ширину провалу −3 дБ (2 Гц в осцилографі, `MAINS_NOTCH_BW_HZ`). Дробове  $N$  використовує лінійну інтерполяцію між двома обхідними цілими відводами (`process`, :345).

- **Відстеження** (`track`, :289): вікнає еталон Ганном, потім сканує смуги 50 Гц і 60 Гц **плюс їхні 2-гі гармоніки (100/120 Гц)** потужністю Герцеля (`scanBand` + параболічне суб-крокове уточнення, :395), авто-визначаючи 50 проти 60 за сумарною енергією  $H1+H2$  і фіксуючись від сильнішої гармоніки ( $H2/2$ , коли домінує 2-га — стійко до випрямленого гулу). Фіксація вимагає, щоб лінія перевершила медіанний сканований базовий рівень на `LOCK_RATIO = 4` (~6 дБ); детекція EWMA-згладжена (`LOCK_SMOOTH = 0.97`), а викиди  $> 0.5$  Гц відкидаються (:316–337).
- **Застосування зі збереженням DC** (`processPreservingDc`, :378): гребінка внутрішньо режектує DC, що обнулило б середнє осцилограми; цей варіант віднімає середнє блоку, гребінкує, потім додає назад, тож знімається лише гул (не DC) — тримаючи `Vmean` осмисленим.

Осцилограф перевідстежує щонайбільше кожні 200 мс (`MAINS_TRACK_PERIOD_NANOS`), `reset()` - ить і перезастосовує гребінку щоперемалювання по суцільному вікну читання, тож її стартовий перехідний процес лишається за лівим краєм екрана у `pre-roll` (`applyMainsSuppression`, `ScopeView.java:1244`). (Це **часова** гребінка; модуль FFT використовує окремий частотний поділ `applySpectrumCorrection`, теж у цьому класі на :203, не використовуваний осцилографом.) Оскільки лінії затримки гребінки стартують обнуленими щопроходу, робітник вимірює **усталений хвіст** ( $\approx 3 \cdot f_s / (\pi \cdot BW)$  відліків углиб, обмежено половиною вікна), а не непридушену голову при обчисленні  $V_{pp}/V_{rms}$  (`ScopeMeasurementWorker.java:431`).

Порядок застосування фільтрів на перемалювання/вимірювання: ВЧ-ФНЧ/де-спайк → мережева гребінка → тригер/малювання/вимірювання (`ScopeView.java:1877`, `ScopeMeasurementWorker.java:382`).

## 2.6 Реконструкція обвідної биття подвійного тону

`reconstructBeatSignal` (`ScopeView.java:2194`) відновлює повільний модулятор  $\cos((F1-F2)/2 \cdot t)$  від  $\sin(F1 \cdot t) + \sin(F2 \cdot t) = 2 \cdot \sin((F1+F2)/2 \cdot t) \cdot \cos((F1-F2)/2 \cdot t)$ , щоб тригер міг зафіксуватись на битті, а не тремтіти між нулями носія:

1. **Випрямлення + каскадований бокскап ФНЧ** — `|data|`, згладжене двома ковзними-середніми боксарами довжини  $L \approx \text{чверть-період-биття}$  у каскаді (відгук  $\text{sinc}^2$ , глибша смуга затримування), кожен видає у центрі свого вікна для нульової сумарної групової затримки ( :2222 – 2244 ).
2. **Синхронне I/Q-детектування** — оскільки `|cos|` розкладається у ряд Фур'є, чия перша АС-гармоніка стоїть на  $(F1-F2)$  з фазою  $2 \cdot \phi_m$ , I/Q-кореляція `absLp - DC` проти `cos/sin(2\pi \cdot \text{beatHz} \cdot i)` дає `phase = atan2(Q,I) = 2 \cdot \phi_m` ( :2271 – 2289 ).
3. **Вивід** чистого `rawPeak \cdot cos(\text{omegaMod} \cdot i - \text{phaseMod})` (частота/фаза модулятора поділені навпіл), узгоджений за піком із піком `|F1|+|F2|` сирого сигналу, тож накладка торкається обвідної носія ( :2290 ). Стандартний тригер потім працює по цій обвідній (`sinc-уточнення` вимкнено), а `drawBeatOverlay` ( :2141 ) за бажанням її малює. Частоти генератора спершу прив'язуються до FFT-біна (`FftBinSnap.snapIfEnabled`), тож реконструйований `|F1-F2|` збігається з тим, що фактично випромінюється (`ScopeView.java:1982` ).

## 2.7 Повноперіодне вікнування та збереження файлу за нульовими перетинами

`ScopeFileSaver` (`ScopeFileSaver.java`) прив'язує збережений діапазон до **зростаючих нульових перетинів на період один від одного**, тож зацикленому плеєру немає клацання на шві. `findFullPeriodWindow` ( :71 ) обчислює `periodSamples = round(sampleRate / freqHz)`, знаходить перший зростаючий нульовий перетин у першому періоді (`firstRisingZeroCross`, :96, умова `arr[i-1] < 0 && arr[i] >= 0`) та останній зростаючий нульовий перетин у фінальному періоді (`lastRisingZeroCross`, :104), і зберігає `[startSnap, endSnap]` — ціле число періодів, обидва кінці на амплітуді  $\approx 0$ , зростаючі. Відкочується до тривіального `(0, actual)` вікна, коли частоти не виявлено. Виявлена частота йде з `ScopeView.getLastFrequencyHz` (`ScopeView.java:633`). Відліки квантуються у знаковий PCM на обраній бітності порціями по 4096 кадрів (`save`, :118).

## 2.8 Математика «гарних чисел» В/поділку та час/поділку (OscParse)

`OscParse` (`OscParse.java`) тримає списки кроків **декади 1-2-5** для В/поділку (1 мкВ/поділку ... 500 В/поділку) та час/поділку (1 мкс/поділку ... 1 с/поділку) і переводить між мітками та базовими одиницями. `parseUnit` ( :137 ) розщеплює числову та SI-префіксну частини ( $\mu/u=1e-6$ ,  $m=1e-3$ , без=1). `voltsPerDivTargets / timePerDivTargets` ( :79, :88 ) надають зростаючі масиви значень для крокування `StepSelector`. `formatWithUnit` ( :112 ) авто-префіксує мантису у `[1, 1000]` і зрізає кінцеві нулі. `tryParseStepInput` ( :160 ) — поблажливий парсер для редагованих полів, що приймає короткі форми як `100m`, `100mV`, `5` (без  $\rightarrow$  базова одиниця), з  $n=1e-9$  аж до без=1; повертає `null` на нечисловий ввід. Масиви міток тримаються вручну синхронними з `MainWindow.VOLT_PER_DIV / TIME_PER_DIV`.

## 2.9 Графіка: масштабування, рендеринг полілінії, децимація, сітка

**Масштабування значення  $\rightarrow$  піксель** — на канал `vScale = peakVolts/vDiv \cdot pixelsPerDivY` із `pixelsPerDivY = h / DIVISIONS_Y`; нульова лінія сидить на `centerY = h \cdot offsetFrac` (повзунок зсуву), а піксель `y = centerY - (sample - dcOffset) \cdot vScale` (`renderTraces`, `ScopeView.java:2357 – 2368`; `drawTrace`, :2436). Часове масштабування: `windowSeconds = timePerDiv \cdot DIVISIONS_X`, `displaySamples = round(windowSeconds \cdot sampleRate)` ( :1836 ).

**Дворежимний рендеринг осцилограм** (`drawTrace`, `ScopeView.java:2415`):

- **Щільний / збільшений** (`samplesPerPx \leq \text{MAX\_LANCZOS\_DOWNSAMPLE} = 5`): один реконструйований Y на піксельний стовпець — `sinc` (Ланцош, масштабований), коли

увімкнено, інакше посеместна лінійна інтерполяція між відліками ( :2438 ) — прокреслено через спільний відсічений Path `paintPolyline` (той самий рендер, що траси FFT/FreqResp), з AA та круглими ковпачками/з'єднаннями. `subSampleOffset` зсуває рендерений сигнал на частку відліку, тож тригер потрапляє на центральний піксель.

- **Розріджений / повільний час-поділок** ( `samplesPerPx > 5` ): **стовпці min/max на стовпець** — для кожного піксельного стовпця сканує покриті відліки і малює вертикальну лінію від min до max ( :2486 ), з вимкненим AA і плоскими ковпачками для швидкості (стан зберігається/відновлюється, щоб крива наступного каналу не стоншилась). `ZoomedView.drawTrace` використовує те ж дворежимне розщеплення ( `ZoomedView.java:215` ).

**Точки відліків** — коли інтервал відліків перевищує 10 px ( `pxPerSample > 10` ), заповнений овал `dotDiameter` накладається на кожен відлік, суб-відлічно зсунутий на `subSampleOffset · pxPerSample` ( `ScopeView.java:2452` ).

**Сітка / графікула** — лінійна сітка поділок 10×10 через спільний `drawGrid` із центрованим перехрестям, що несе `TICKS_PER_DIV` проміжних поділок ( `paintCanvas` , `ScopeView.java:728` ). `ZoomedView` малює власну сітку + середню лінію ( :126 ). Повзунки — пунктирні перехрестя із заповненими трикутними ручками ( `drawSliders` , :1359 ); лінії меж  $\pm FS$ , мітки напруги/часу на краях ( `drawEdgeLabels` , :1643 ) та таблиця вимірювань довершують накладку.

**Антиаліасинг/післясвічення** — криві використовують AA `SWT.ON`; сітка і стовпці `SWT.OFF` (різко). **Післясвічення/фосфорного спаду немає** — кожне перемалювання перерендерує один кадр. Заморожений знімок `RenderedFrame` (вікнові відліки + параметри рендеру, :1151 ) дозволяє панелі знімка відтворити точну екранну (можливо заморожену) осцилограму.

## 2.10 Модель потоків захоплення та буферизації

Подання читає спільне кільце `SignalBufferReader` відносно його `writePos` через `readEndingAt / readLatest`; жоден курсор читання не мутується, тож живе захоплення і осцилограф ніколи не конкурують ( `ScopeView.java:96` ). Робітник вимірювань ( `ScopeMeasurementWorker` ) запускає демон-потік на фіксованому **10 Гц** ритмі з компенсацією дрейфу ( `MEAS_COMPUTE_PERIOD_NANOS` , :52 ; цикл :279 ), тож вибірки `avg/min/max/σ` рівномірно рознесені; обмежує кожне читання `MEAS_MAX_SAMPLES = 96000` ( :59 ) і виключає 500-мс префікс `AC_WARMUP_NANOS` , щоб уникнути стартового перехідного процесу ADC, що змістив би опубліковане DC-середнє ( :362 ). Результати лягають у 1024-глибоке кільце історії ( `measHistory` , :107 ), захищене `measHistoryLock`; потік перемалювання читає знімки і обходить кільце під тим замком ( `walkRecentHistory` , :183 ; `averagedChannelMean` , :200 ). Віднімання DC при AC-зв'язку використовує  $\geq 500$ -мс-усереднене середнє на канал ( `AC_DC_MIN_AVG_NANOS` , `ScopeView.java:295` ), тож осцилограма і тригер не хитаються між тактами робітника. `cap/s` — це EMA інтервалу між новими кадрами, спадаючи до нуля на заморожених кадрах ( `updateCaptureRate` , :770 ).

---

## 3. FFT-аналізатор

Цей розділ документує кожен DSP-, математичний, спектральний, усереднювальний і графічний алгоритм у модулі FFT-аналізатора. Модуль обчислює живий когерентно-усереднений спектр і виводить THD / THD+N / SNR / SINAD, гармоніки, IMD, робастний рівень шуму та придушення мережі. Посилання — `file:line`.

### 3.1 Власне FFT — Кулі-Тьюкі основи 2, DIT, на місці, паралельне



`fft/Fft.java` — це FFT Кулі-Тьюкі основи 2 із **проріджуванням за часом** без стану, що працює на місці на окремих масивах `double[] re, im`, чия довжина має бути степенем двійки (`Fft.java:30-48`, `forward()` на `:74`).

- **Перестановка з реверсом бітів** (`bitReverse`, `:88`) переупорядковує вхід реверсом бітів індексу через стандартний інкрементний трюк із переносом, обмінюючи `re[i]/im[i]` з `re[j]/im[j]` для  $i < j$ . Тримається послідовною (дешево).
- **Стадії «метеликів»** (`stageGroups`, `:138`): для довжини стадії  $len = 2, 4, \dots, N$  кожна група  $[i, i+len)$  виконує  $len/2$  «метеликів» з поворотним множником  $W = e^{-j \cdot 2\pi / len}$ , обчисленим раз як `cos(ang)`, `sin(ang)` ( $ang = -2\pi / len$ , `:79`) і просунутим інкрементно на «метелик» через комплексну рекурентність `cur ← cur · W` (`:155-157`). Кожен «метелик» — канонічна пара  $u \pm W^k \cdot v$  (`:147-154`).
- **Паралелізм** (`butterfliesParallel`, `:106`): у стадії  $n/len$  груп торкаються неперетинних зрізів, тож для  $N \geq PARALLEL\_THRESHOLD = 65536$  (`:51`) групи розбиваються між демоном-пулом потоків (`THREADS = cores-1`, `:53`) з `invokeAll` у ролі бар'єра на стадію (`:127`). Нижче порогу, або на  $\leq 2$ -ядерних машинах, лишається послідовним. Результати біт-ідентичні послідовному шляху — різняться лише розбиття.
- **Обробка реального входу**: аналізатор подає реальний сигнал із `im[] = 0` і далі використовує лише одnobічні спектральні біни  $0 \dots N/2$ ; IFFT для вікна Долфа-Чебишова експлуатує  $IFFT(W) = FFT(W)/N$  для реального  $W$  (див. §3.4).

## 3.2 Вікнування та когерентне підсилення

`FftAnalyzer.buildWindow` (`fft/FftAnalyzer.java:1556`) будує вікно аналізу довжини  $N$ ; таблиця кешується і перебудовується лише при зміні  $(N, windowType)$  (`getCachedWindow`, `:130`), щоб уникнути  $\sim 1$  млн обчислень `cos/sinh` при великому  $N$ . Типи вікон (`enums/WindowType.java`):

- **RECT** — усі одиниці.
- **HANN** —  $0.5 \cdot (1 - \cos(2\pi n / (N-1)))$ .
- **BH4** — мінімальний 4-членний Блекмен-Гарріс (Harris 1978), коефіцієнти `0.35875`, `0.48829`, `0.14128`, `0.01168`.
- **BH7** — 7-членний Блекмен-Гарріс (Nuttall 1981), косинусна сума зі змінним знаком.
- **FT** — 5-членний flat-top SRS SR785, **періодична** нормалізація  $N$  для точності амплітуди (всі інші використовують симетричне  $N-1$ ).
- **HFT144D / HFT248D** — high-flattop вікна Heinzel (бічні пелюстки 144 / 248 дБ, плоска амплітуда; з каталогу коефіцієнтів звіту GH\_FFT).
- **KB24 / KB38** — вікна Кайзера  $w[n] = I_0(\beta \cdot \sqrt{1-x^2}) / I_0(\beta)$  при  $\beta = 24$  ( $\approx -226$  дБ бічних пелюсток, вужча головна пелюстка за рівнопридушувальні косинусні вікна) і  $\beta = 38$  (нижче FFT-підлоги подвійної точності).
- **DC150 / DC200 / DC250 / DC300** — вікна Долфа-Чебишова при придушенні бічних пелюсток 150–300 дБ (§3.4).

**Когерентне підсилення** обчислюється як `cohGain = (Σ w[n]) / N` (`:444-448`) і вкладається у нормалізацію амплітуди `normFactor = 1 / (N · cohGain)` (`:1106`), тож вікнований тон читає свою справжню амплітуду. Однобічний спектр множить не-DC/Найквіст біни на 2 (`:1138-1140`). `MathUtil.chebyshevT/acosh` (`fft/MathUtil.java:38,49`) постачають поліном Чебишова (тригонометрична форма для  $|x| \leq 1$ , гіперболічна поза) для DC-вікон.

## 3.3 Обробка перекриття

`enums/FftOverlap.java` визначає частки перекриття 0 / 50 / 75 / 87.5 / 93.75 %. Хоп — `step = round(N · (1-fraction))` (`FftAnalyzer.java:423`), а кількість кадрів — `(len-N)/step + 1` (`:424`). Це



**аналіз із перекриттям** (послідовні вікновані FFT, що поділяють відліки), а не ресинтез overlap-add. GUI-робітник зсуває своє утримуване вікно аналізу вперед точно на один хоп за такт ( `FftAnalyzerWorker.java:1424-1436` ), тож область перекриття переюзується без перерасчетування кільця захоплення.

Практична примітка: високе перекриття обмежене даними — 93.75 % дає 16 кадрів, що поділяють 93.75 % своїх відліків (корельований шум), тож лічильник «N усереднень» біжить  $\approx 16\times$  ефективних незалежних усереднень, а рівень шуму спадає на швидкості даних за секунду незалежно від перекриття.  $\sim 75$  % відновлює крайовий спад вікна; понад те — здебільшого зайве FFT-обчислення для тієї ж підлоги за секунду.

### 3.4 Вікно Долфа-Чебишова через обернене DFT

`buildChebyshevWindow` ( `FftAnalyzer.java:1631` ) будує рівнопультасійне вікно для цільового придушення бічних пелюсток `attenDb`:

1.  $\beta = 10^{(\text{atten}/20)}$ ,  $x_0 = \cosh(\text{acosh}(\beta)/(N-1))$  ( :1632 ).
2. Частотні вибірки  $W[k] = T_{N-1}(x_0 \cdot \cos(\pi k/N))$ , заповнені симетрично ( :1639 ).
3. Часове вікно через  $\text{IFFT}(W) = \text{FFT}(W)/N$  (реальне  $W$ , :1649).
4. `fftshift` на  $N/2$ , щоб центрувати головну пелюстку ( :1657 ).
5. Нормалізація на пік 1 ( :1662 ).

### 3.5 Суб-бінна оцінка частоти (`kFractional`) і синтез гармонік

Справжня частота тону —  $k_f \cdot F_s/N$  з нецілим  $k_f$ ; використання цілого біна для деротації гармонік спричиняє посеместний дрейф фази  $2\pi \cdot \epsilon \cdot n/N$  і повне скорочення на довгих записах ( :451-457 ).

- **Оцінювач за різницею фаз** ( :780-792 ): два кадри на `step` відліків один від одного дають  $k_f = (\Delta\phi - 2\pi m) \cdot N / (2\pi \cdot \text{step})$  після дезамбігуації цілих циклів  $m = \text{round}((\Delta\phi - \text{expected})/2\pi)$  — незалежно від вікна, точно до  $\sim 1e-5$  біна.
- **Параболічна інтерполяція** як резерв ( `MathUtil.parabolicBinInterp`, `MathUtil.java:61` ): тритчкова лог-степенева парабола  $\delta = 0.5 \cdot (\alpha - \gamma) / (\alpha - 2\beta + \gamma)$ , коли є лише один чистий кадр ( :794 ).
- **Перепідгонка к на довгій базі** ( `refineKappaOverSegment`, :236 ): регресує **нахил** деротованої фази основи по кожному чистому кадру через однобінний **Герцель** на `refIntBin` ( $O(N)/\text{кадр}$ ,  $\sim 6$  % повного FFT) замість одного хопу. Залишки рецентруються на їхньому **циркулярному середньому** (безпечно до завертання), а зсуви центруються для обумовленості; нахил дає  $k$  з базою  $(\text{bestLen}-1) \cdot \text{step}$  ( :260-272 ), стискаючи залишковий дрейф  $\sim u$  той множник. Застосовується лише для когерентного усереднення з  $\geq 4$  чистими кадрами і обмежується перевіркою до  $|\Delta k| < 0.5$  ( :863-877 ).
- **Виявлення гармонік** ( `detectHarmonics`, :1753 ): фізика приколює  $N$ -ту гармоніку до дробового біна  $N \cdot k_{\text{Fractional}}$ ; два обхідні біни **зважаються за потужністю** суб-бінним зсувом  $w = |\text{offset}| \in [0, 0.5]$ :  $|X|^2 = (1-w) \cdot |X[\text{main}]|^2 + w \cdot |X[\text{adj}]|^2$  ( :1778-1781 ), відновлюючи енергію, розщеплену витоком вікна, замість пошуку локального максимуму, що чіпляється за шум.
- **Оцінка другого тону**: той самий метод різниці фаз/параболи по чистому кадру працює навколо підказаного чи авто-виявленого другого тону ( :802-855 ), тож показники подвійного тону рапортують справжні суб-бінні частоти, а не піки з колапсованого середнього.

### 3.6 Когерентне (векторне) проти некогерентного (потужність) усереднення та пологова деротація

Два режими усереднення (прапор `coherentAveraging`, `FftAnalyzer.java:594`):

- **Когерентне** — сума комплексних спектрів після деротації кожного кадру до спільного часового початку; шум (випадкова фаза) скорочується, SNR покращується  $\sim \sqrt{\text{кадрів}}$  (:1127-1131).
- **Некогерентне** — сума побінної потужності  $|X[k]|^2$  (без вирівнювання фази), `mag=√(sum/fc)` (:1132-1136).

Ключовий когерентний алгоритм — **пологова деротація сталою фазою** (Прохід 2, :953-1042): міжкадровий приріст фази основи —  $\Phi = -2\pi \cdot k_f \cdot f \cdot \text{step} / N$  (:986); кожен бін прив'язується до найближчої гармоніки  $h = \text{round}(\text{signed-bin} / k_0)$  і деротується **сталою** фазою цієї пелюстки  $h \cdot \Phi$  (кешовано через інкрементний фазор; негативне  $h$  — спряжене, :991-996). Критично, це **не** побінний частотний нахил — нахил збігається лише на точних бінах гармонік і пере-обертає спідницю витоку  $\alpha$  (зсув-біна  $\times$  кадр), продукуючи гребінку Діріхле/array-factor на  $F_s / (\text{frames} \cdot \text{step})$  на спідниці. Стала фаза на пелюстку вирівнює всю пелюстку, убиваючи гребінку (:960-970).

### 3.7 Деротація подвійного тону / IMD-сітки

Для справжнього другого тону одноеталонна деротація прив'язала б  $F_2$  до гармоніки  $F_1$  і могла б скоротити її на  $180^\circ$  (баг подвійного тону IMD). Аналізатор пробує обидві деротації  $h \cdot \Phi(F_1)$  і  $\Phi(k_F2)$  по сегменту і лишає ту, що лишає  $F_2$  сильнішим (:879-911); неправильна скорочується, тож найсильніша само-обирається. Далі він будує **IMD-сітку продуктів**  $a \cdot k_{F1} + b \cdot k_{F2}$  ( $|a|+|b| \leq \text{order}$ ,  $b \neq 0$ ) і деротує кожен пелюстку продукту на  $a \cdot \Phi(F_1) + b \cdot \Phi(F_2)$  по  $\pm 24$ -біновій пелюстці (:912-951, :997-1019), тож уся 2-тонова гребінка деротується на своїх справжніх частотах. Члени  $b=0$  лишаються одноеталонними.

### 3.8 Відкидання кадрів (наразі вимкнено) та вибір чистого сегмента

Покадрове відкидання за R-інваріантом / фазовою когерентністю підключене, але застролено вимкненим (`frameRejection=false`, :530); коли увімкнено, працює так:

- **Піфагорів синусний інваріант** (:499-523): для  $s=A \cdot \sin(\theta)$ ,  $R=\sqrt{(s^2 + (\Delta s/k)^2)} \equiv A$ ; чисті відліки дають  $R \approx A$ , збої відхиляються. Використовує центральну різницю, працює на сигналі зі знятим DC (`sampleMean/dcRemovedRms`, :1715,1724), з `peakAmp=√2·RMS`. Влучання відліків групуються в події (проміжок  $\leq 16$  відліків), а перекривні кадри маркуються брудними (:610-690).
- **Контролі здійсненності**: пропускається, коли синус закопано  $>20$  дБ нижче часового піка (:540-543), коли відношення похідного шуму  $\text{RMS}(\Delta^2 s) / (A \cdot \omega^2 / \sqrt{2}) > 5$  (`derivativeNoiseRatio`, :1969), для багатотонових сигналів, або коли всі кадри відкинулись би (:558-708).
- **Перевірка фазової когерентності** (:1044-1101): після корекції кадри, що відхиляються від **циркулярно-медіанної** фази основи (`circularMedianPhase`, :1948 — медіана  $\sin/\cos$ -компонент одиничного кола) на  $>3^\circ$ , перераховуються FFT і віднімаються з акумулятора.
- **Найдовший чистий прогін** (:732-743) обирає суцільний сегмент чистих кадрів, використаний для переоцінки  $k$  та накопичення.

Жива міжтактова страхувальна сітка проти збійних кадрів — натомість спектральний `SpectralDiscontinuityDetector` (§3.13), а часовий відкидач  $\Delta^n$  тримається, але вимкнений.

### 3.9 Одиниці магнітуди та еталонна математика

**dBFS — єдиний базовий масштаб.** `FftResult` несе побінні амплітуди лише у dBFS (без dBV-копій; уточнений рівень основи — `fundamentalTrueDbFs`). `enums/MagnitudeUnit.java` переводить dBFS у показану одиницю на час побудови (`convertFromDbFs`):

- **dBFS** — наскрізно.
- **dBV** —  $\text{dbFs} + \text{dbvOffsetDb}$ , де  $\text{dbvOffsetDb} = 20 \cdot \log_{10}(\text{adcFsVoltageRms})$  (кешовано у `Preferences`, перераховується при зміні калібрування ADC) прив'язує 0 dBFS до абсолютної напруги.
- **V (Vrms)** —  $10^{(\text{dBV}/20)}$ .
- **V/√Hz** —  $V / \sqrt{\text{binBw}}$  (PSD,  $\text{binBw} = F_s/N$ ).

Оскільки збережені дані ніколи не несуть вольтів, перекалібрування ADC миттєво перемічує кожне показане значення, не торкаючись результатів. Знаменник відношення `refLin` для THD/SNR/THD+N/гармонік % — це виміряна основа, або задана користувачем ручна основа (введена у V/dBV, переведена раз у dBFS) — ручна основа впливає лише на еталонний рівень, ніколи на спектр. `FftFormat.magToYFraction` відображає значення у вертикальну частку, лог-масштабовану для осей V/√Hz і лінійну для дБ-осей (`gui/fft/FftFormat.java`).

### 3.10 Рівень шуму (median + MAD-стиль), маскування сигналу, THD / THD+N / SNR / SINAD

- **Маска сигнальних бінів** (`buildSignalBinMask`, :1796): основа + кожен бін гармоніки, кожен розмазаний на  $\pm \text{EXCL\_BINS}=4$  для витоку вікна; виключаються і з медіани шуму, і з суми SNR.
- **Рівень шуму** (`computeNoiseFloorAndExtendSignalMask`, :1834): дві медіани за одне сканування — **обмежена-діапазоном медіана** (поріг сплеску / SNR) і повноспектральна **10-та процентиль** підлоги (несприйнятлива до витоку, ближча до справжнього шуму квантування, ніж медіана, що піднята спурами, :1867-1870). **Динамічний обхід виключення основи** розширює маску назовні від основи, поки біни перевищують 10-ту процентиль (:1876-1908); **зона фазового шуму** в межах  $\pm \text{fundBin}/2$  маркує будь-який бін над уточненою медіаною як сигнал (:1910-1924). Сорткування використовує `Arrays.parallelSort`.
- **THD** =  $\sqrt{(\sum \text{harmonic}^2) / \text{refLin}} \times 100\%$  по H2..H9 у смузі спотворень (:1273-1288).
- **SNR** =  $10 \cdot \log_{10}(\text{refLin}^2 / \sum \text{noise-bin power})$  по не-сигнальних бінах у смузі (:1315-1329).
- **SINAD** =  $10 \cdot \log_{10}(\text{refLin}^2 / (\text{noisePower} + \text{harmPowerSum}))$ ; **THD+N = -SINAD**; ENOB виводиться з SINAD як  $(\text{SINAD}-1.76)/6.02$  (:1331-1345). `recomputeStats` (:1418) перевиводить усе це з (можливо постобробленого / накопиченого) `amplitudeDbFs`, тримаючи позиції бінів фіксованими.

### 3.11 IMD-аналіз (конвенція різницевих частот CCIF/DIN)

`gui/fft/ImdAnalyzer.java` — чиста функція над спектром **dBFS** (`amplitudeDbFs`; відношення тонів/продуктів незалежні від масштабу, тож жодне калібрування напруги не входить у математику), що продукує `ImdResult` (`gui/fft/ImdResult.java`):

- **Виявлення тонів** —  $\arg\max$  у межах  $\pm 8$  бінів кожної командної частоти, уточнений 3-точковою квадратичною інтерполяцією піка  $\delta = 0.5 \cdot (y_L - y_R) / (y_L - 2y_C + y_R)$  зі значенням інтерпольованого піка за Смітом і Серрою  $y_C = 0.25 \cdot (y_L - y_R) \cdot \delta$  ( `refinePeak`, :211-237). Справжні суб-бінні частоти йдуть з оцінок аналізатора по чистому кадру, коли доступні ( :101-104).
- **Продукти** ( :126-152):  $d2L = f2 - f1$ ,  $d2H = f1 + f2$ ; для порядку  $n \geq 3$ ,  $dnL = (n-1)f1 - (n-2)f2$  (нижче  $F1$ ) і  $dnH = (n-1)f2 - (n-2)f1$  (вище  $F2$ ), порядки 2..5. Плюс **DFD2** =  $f2 - f1$  і **DFD3** =  $\sqrt{(|F(2f1 - f2)|^2 + |F(2f2 - f1)|^2)}$  ( :118-124).
- **Відношення** — еталон — це **лінійно-магнітудна сума**  $|F1| + |F2|$  ( $V_{rms}$ ); кожен продукт виражається як % від неї; `imdPwrPct` — RMS-сума всіх продуктів над тим же знаменником ( :113-155).
- **TD+N** ( :157-202 ) — за Парсевалем/незалежно від вікна:  $100 \cdot (V_{rms} - \sqrt{F1^2 + F2^2}) / V_{rms}$ , де  $V_{rms}^2$  сумує квадрати побінних напруг по всьому спектру, а член  $F1/F2$  сумує по **динамічній спідниці** кожного тону (обхід від піка до 10-ї процентилі підлоги, `noiseFloorDbV/skirtEdge`, :250-281). ENBW вікна скорочується у відношенні, тож це правильно для будь-якого вікна.

### 3.12 ToneLobeLift — лог-домене розтягнення пелюстки (приколоти ноги до шуму, не рівномірний підйом)

`dsp/ToneLobeLift.java` перемасштабовує головну пелюстку тону (для `.frc`-калібрування  $1/|N|$  чи ручного рівня основи) як **вертикальне розтягнення, прив'язане до рівня шуму**, продукуючи плавний купол, чії ноги лишаються на траві — явно **не** рівномірний підйом (що дав би плосковерху коробку з вертикальними скелями, :24-54):

1. **Робастна стеля підлоги** (`localFloor`, :80):  $\text{median} + 4 \cdot \text{MAD}$  по широких фланкових смугах (`floorGap/floorSpan`), тож широка пелюстка / гармоніки / мережа / спури, що потрапляють усередину, ігноруються як викиди.
2. **Протяжність пелюстки** (`lobeBins/edge`, :103,108): обхід від піка, поки бін не впаде до підлоги; біновий пік завжди включено.
3. **Розтягнення** (`stretch`, :127):  $|X'| = |X| \cdot \text{factor}^t$  з  $t = \ln(|X|/\text{floor})/\ln(\text{peak}/\text{floor})$  ( $t=1$  на піку,  $0$  на підлозі).  $t$  обмежено 1, тож шумовий бін, що стоїть над припущеним піком, не підіймається більше за пік ( :133-136). Біни на/нижче підлоги недоторкані; суб-підлоговий пік просто масштабується. `FftView.stretchLobe` (`gui/fft/FftView.java:1626`) застосовує це на тон на час побудови.

### 3.13 SpectralDiscontinuityDetector — 3-вентильний відкидач збоїв за ковзною медіаною

`dsp/SpectralDiscontinuityDetector.java` відкидає збійні/застряглі блоки **перед** тим, як вони ввійдуть у міжтактове векторне усереднення, порівнюючи спектр кожного блоку з поточною статистикою прийнятих блоків (справжня лінія само-референтна; збій підіймає підлогу чи приносний п'єдестал). Кожне порівняння — самокаліброване дБ-відношення ( $\text{median} + k \cdot \text{MAD}$ , `mad` масштабовано  $\times 1.4826 \approx \sigma$ , :420-427). Перший блок засіває еталон і приймається безумовно ( :205-213). Смуги **лог-рознесені** (`buildBands`, :344), звужені до  $\pm \text{FLOOR\_GUARD\_HZ}$  навколо кожної основи.

- **Вентиль 1 — приносний п'єдестал** ( :215-223 ): медіанна потужність спідниці, що фланкує кожну основу (виведену з даних головну пелюстку виключено через `ToneLobeLift`, `pedestalDb` :275) мінус локальна підлога; надлишок незалежний від амплітуди/витоку, застроблений проти поточного  $\text{median} + 10 \cdot \text{MAD}$  (більша  $\sigma$  для важкого хвоста витоку).

- **Вентиль 2 — широкосмуговий підйом підлоги** ( :225-240 ): еталон поточної медіани на смугу; середній підйом по не-лінійних смугах  $> \text{median} + 6 \cdot \text{MAD}$  відкидає рівномірний широкосмуговий підйом, який п'єдестал не бачить.
- **Вентиль 3 — повна потужність** ( :242-246 ):  $|\text{powerDb} - \text{median}| > 6 \cdot \text{MAD}$  ловить застрягання генератора / довге пропадання, де лінії колапсують замість підняття підлоги.

Підлоги MAD (  $\text{MIN\_MAD} = 0.5$ , :64-66 ) не дають дуже-стабільному прогону колапсувати поріг. Знімок `Gates` живить дебаг-накладку ( :441 ). У робітнику відкидання запускає пересинхронізацію ( `onSignalDiscontinuity`, `FftAnalyzerWorker.java:526` ).

### 3.14 Міжтактове когерентне накопичення, контури фазової фіксації та FLL

Робітник ( `gui/fft/FftAnalyzerWorker.java` ) запускає глибше міжтактове усереднення, ніж один виклик `analyze()`. Спектр кожного такту обертається до приколеного еталонного часового початку і зважується кадрами; режим `forever` — це справжнє кумулятивне середнє (SNR  $\sqrt{\text{загальних-кадрів}}$ ), скінченне кільце  $N$  — експоненційне вікно  $\alpha = \text{weight}/N$  ( `accumulateIntoForeverBuffer`, :595 ).

- **Маршрутизація тонів.** `detectStrongTones` тримає пік як окремий тон лише якщо у межах `fftStrongToneRelDb` (Preferences ► FFT, типово 100 дБ) від *найсильнішого* піка і вище  $\sim 10$  Гц DC-режекторної підлоги. 0-1 тонів  $\Rightarrow$  одноеталонний шлях (тісніше одноразове к-уточнення, гармоніки їдуть на одному нахилі);  $\geq 2$  тонів  $\Rightarrow$  багатотоновий шлях. Чистий тон із далеко-нижчими гармоніками має лишатись одноеталонним.
- **Одноеталонний шлях** ( :686-816 ): приколює `round(kFractional)`, робить **одноразове к-уточнення** з нахилу фази деротованої основи  $d\phi/d\Delta = 2\pi \cdot \Delta k/N$  по `KAPPA_MEAS_FRAMES`  $\approx 24$  чистих кадрах ( $\sim 100\times$  тісніше за однокадровий пік, :706-740), потім запускає неперервний **фазостежний PLL**, що вимірює поточне розузгодження проти глибоко накопиченої фази ( $\sqrt{N}$ -усередненого еталона) і вкладає `PHASE_TRACK_GAIN` від нього у `accumDroppedSamples` ( :741-773 ). Це неперервно перефіксується, убиваючи повільний дрейф к-залишку, який заморожена к заналаштувала б у деротовану фазу (  $2\pi \cdot \Delta k \cdot \Delta/N$ , гармоніки  $h \times$  швидше). Пересинхронізація (переповнення/розрив) — це просто великий залишок  $\rightarrow$  одноразове повне перевирівнювання (підсилення 1), далі контур прибирає. Далі йде полобова деротація сталою фазою по півспектру ( :774-816 ). Захист від пропадання тримає акумулятор, коли основа падає нижче `DROPOUT_FRACTION` від середнього ( :661-669 ).
- **Багатотоновий шлях** ( `accumulateMultiTone`, :853 ): кожен тон (виявлений зі спектру, `detectStrongTones` :1041, з параболічним уточненням і відкиданням гармонік) деротується власною сталою фазою, кожен несе власний потоновий PLL; потоновий залишок  $> \text{PHASE\_JUMP\_RAD} \approx 60^\circ$  запускає одноразове перевирівнювання. Об'єднаний дрейф годинника б деротує не-тонові біни, а IMD-сітка  $a \cdot k_1 + b \cdot k_2$  їде на відстежуваних фазах тонів ( :943-1005 ). Власне к-уточнення вкладає  $\Delta k$  кожного тону раз ( :1006-1031 ).
- **overlayAccumulatorOnto** ( :1141 ) перебудовує показаний дБ-спектр із сирого кумулятивного середнього, виводячи множник `mag`  $\rightarrow$  амплітуда з живого результату, тож перетворення застосовується раз.
- **DerotationPhaseLock** ( `fft/DerotationPhaseLock.java` ) — окремий, юніт-тестований фіксатор **виміряй-потім-скоригуй**: тримає к фіксованою по вікну, вимірює нахил деротованої фази  $2\pi \cdot (k_{\text{true}} - k)/N$ , застосовує одну повну корекцію  $k \pm \text{slope} \cdot N/2\pi$  ( :103-134 ), і оголошує фіксацію після достатньої кількості вікон під `lockDriftRad` (або резерв `maxWindows`). Безумовно стабільний там, де наївний потактовий PLL розходиться.

- **FrequencyFll / FLL** (`gui/fft/FrequencyFll.java`) спрямовує **генератор** на сітку бінів одно-кроковою `deadbeat correction -= (detected - target)` плюс **точне транспортне облікування**: кожна видана корекція записує абсолютну позицію захоплення, з якої вимірювання може її відобразити (`publish-time writePos + 0.7-с захист зливу DAC`), і кожне оновлення, чиє вікно аналізу *починається* раніше, тримається безумовно — таке вимірювання доказово передує корекції, тож дія по ньому наклала б другу корекцію на похибку, що вже у польоті. Це замінює раніші евристичні транспортні моделі (лічильники оновлень, частки прибуття, виміряні мертві часи), які могли неправильно вимірювати під шумом тонкого трекінгу і перекоригувати швидше за фізичну ~3-4-с затримку контуру. У сталому стані тримається в межах смуги `LOCK_PPM = 0.01 ppm` (власна джитер-підлога вимірювання), а **передбачення швидкості дрейфу** (EWMA-відстежуваний нахил Гц/с залишкової похибки — відносне блукання двох вільнобіжних кварців конвертерів, з перевіркою на здоровий глузд) застосовується передбачувально на кожному оновленні, тож похибка лишається біля нуля замість пилкоподібного руху до краю смуги раз на транспортний обхід. `FrequencyAligner/FrequencyAlignerFactory` підключають його як єдину реалізацію `AlignGenerator.FLL`.

Увесь потактовий одноеталонний конвеєр паралелізований (порціонна деротація фазором із неперетинним записом + `Arrays.parallelSort`), тримаючи такт великого FFT під бюджетом хопу 93.75 %-перекриття. Коалесцентна однослотова передача в UI (`AtomicReference<FftResult>`) обмежує бек-лог перемалювання одним спектром, тож швидкий робітник не може накопичити спектри у чергу `asyncExec` SWT і вирости до ООМ. `.frc`-калібрування і мережева корекція застосовуються на **час побудови** на сирому акумуляторі (побінний лінійний множник комутує з деротувати-і-сумувати), тож перемикання калібрування ніколи не псує поточне середнє. Глибоке усереднення працює з частотним *вирівнюванням ВИМКНЕНИМ* (контур вирівнювання весь час рухає захоплену частоту, що деротація не може відстежити).

### 3.15 Придушення мережі в частотній області (гребінкова корекція на час побудови)

`enums/MainsSuppression.java` обирає `NONE` чи `IIR_COMB`. Гребінка **не** застосовується до часових відліків (вхідна IIR-гребінка згортає свій перехідний процес/фазу у кожен кадр, що когерентне середнє не може скасувати); натомість робітник лише **відстежує** мережеву `f0` — кожен 5-й такт аналізу (`MAINS_TRACK_TICK_INTERVAL`; фіксація — EWMA з  $\approx 33$ -детекційною пам'яттю, тож зміна ритму незначна, поки  $\sim 208$  розгортка Герцеля лишаються поза бюджетом більшості тактів) — і нормований частотний відгук гребінки ділиться з показаного усередненого спектру на час побудови (`dsp/MainsCombFilter.java`). `magnitudeAt (:174)` — замкнена форма  $H(z) = (1 - z^{-N}) / (1 - \alpha \cdot z^{-N})$  на одиничному колі ( $\omega N = 2\pi \cdot f / f_0$ ), пік-нормована на  $(1 + \alpha) / 2$ , тож смуга пропускання не зміщена ( $\approx 1$  у смугі,  $\rightarrow 0$  на кожному  $k \cdot f_0$ ). `applySpectrumCorrection (:203)` додає кешовану, смугово-обмежену (`CORR_MAX_HZ`), обмежену знизу побінну дБ-корекцію на місці, перебудовану лише коли `f0` рухається. Акумулятор лишається сирим — аналогічно `.frc`-калу, що так само застосовується на час побудови.

### 3.16 Спільна побудова графіків частотної області (AbstractMeasurementView)

gui/common/AbstractMeasurementView.java — спільний графічний рушій, переюзований і поданням FFT, і Частотної характеристики; gui/common/AbstractFreqDomainView.java додає специфіку лог-частоти.

**Генерація «гарних» поділок осі** (drawGrid, :539):

- **LINEAR** — divisions+1 рівнорознесених поділок (linearTicks, :862).
- **LINEAR\_NICE** — основні на  $1/2/2.5/5 \times 10^n$  (niceLinearMajors, :947: обрати крок з мантиси range/targetCount), із заданим викликом проміжним кроком (niceLinearMinors, :967).
- **LOG** — декадні основні  $10^n$  (logMajorTicks, :871) з  $2..9 \times 10^n$  проміжними (logMinorTicks, :891); мітки адаптивно проріджуються за кількістю видимих декад (adaptiveLogLabels, :1122). Суб-декадне збільшення (isSubDecade, :810) відкочується до гарно-лінійних поділок/проміжних (subDecadeMinors, :827). Розміщення міток піксельно-обізнане і двопрорідне — декадні значення зарезервовані першими, тож кругла декада ніколи не проріджується (:704-716) — з пропуском колізій за текстовою протяжністю.

**Відображення значення→піксель:**

- axisFraction (:930) повертає позицію [0,1] — лог використовує основу-10 ( $\log_{10}(v) - \log_{10}(\min) / (\log_{10}(\max) - \log_{10}(\min))$ ), що збігається з freqToX; valueToX/valueToY відображають у абсолютні пікселі (Y інвертовано, :915,922).
- AbstractFreqDomainView.freqToX/xToFreq (:164,184) — це перетворення  $\text{freq} \leftrightarrow X$  із  $\text{safeMin} = \max(1, \text{freqMin})$  і крихітним мультиплікативним захистом safeMax; dbToY (:227) лінійно-дБ; magToY/magToYTrace (:236,249) маршрутизуються через FftFormat. magToYFraction (затиснуто для маркерів, незатиснуто для траси, тож поза-діапазонні нахили лишаються правильними, а кліп GC зрізає їх врівень).

**Рендеринг полілінії спектру** (ColumnBucketPainter, :1188 через paintPolyline, :1162): розкладає до тисяч бінів у  $\leq \text{plot.width}$  піксельних стовпців. Прохід 1 малює один **вертикальний стовпець обвідної** на багатоточковий стовпець (AA вимкнено, Y затиснуто до plot, тож межа GDI  $\pm 32k$  не перевищується, :1247-1259); прохід 2 прокреслює єдиний антиаліасований Path середніх точок через центри стовпців, кожен сегмент **відсічено за Лянгом-Барскі** до plot (clipSegmentToPath, :1301), тож далекі координати ніколи не завертаються, з лівим/правим крайовими якорями, тож траса досягає країв plot. SKIP\_Y відкидає NaN-відліки.

**Інші спільні засоби:** paintCachedStatic (AbstractFreqDomainView.java:106) кешує статичний шар (сітка+осі+траси), ключований відбитком long, тож перемалювання перехрестя/блимання не переобходять дані; drawReadoutBox (:276) — авто- перевертний зчит перехрестя; drawOutlinedText/drawOutlinedDashedLine (:210,256) дають піковим міткам і маркерам ореол кольору фону над живою трасою.

---

## 4. Частотна характеристика

---

Модуль Частотної характеристики вимірює передавальну функцію  $H(f)$  пристрою-під-тестом — амплітуду (дБ) та фазу (градуси) проти частоти — по всій аудіо-смузі. На відміну від класичного покрокового синусного свіпу, цей модуль використовує **єдиний неперервний**



**експоненційний лог-свіп Фаріни**, відтворений один раз, захоплений на обох каналах ADC і переведений у  $H(f)$  **деконволюцією в частотній області** ( $H = Y/X$ ). Обидва стерео-канали деконволюються паралельно з одного відтворення.

Файли основного конвеєра: `FreqRespAnalyzer.java` (оркестрація), `FreqRespCalHelper.java` (DSP), `SignalGenerator.java` (генерація свіпу), `FreqRespView.java` (побудова + калібрування /RIAA на час малювання).

## 4.1 Генерація експоненційного лог-свіпу Фаріни

**Алгоритм.** Експоненційний синусний свіп (чірп) одиничної амплітуди,  $x(t) = \sin(K \cdot (e^{t/L} - 1))$  із  $L = T/\ln(f_1/f_0)$  і  $K = 2\pi \cdot f_0 \cdot L$ . Миттєва частота просувається *лог-лінійно* від  $f_0$  при  $t=0$  до  $f_1$  при  $t=T$ ; фаза стартує точно з 0, тож свіп конкатенується з провідною тишею без розриву. Рендериться раз у буфер `float[]` (`SignalGenerator.renderLogSweep`, `SignalGenerator.java:367-377`; побудова на `SignalGenerator.java:336-354`).

**Чому експоненційний (Фаріна), а не покроковий синус.** Єдиний неперервний свіп збуджує кожную частоту за один прохід, даючи значно кращий час-вимірювання/SNR, ніж затримування на  $N$  дискретних тонах, а експоненційний профіль витрачає рівну енергію на октаву — узгоджуючись із лог-частотним показом та людським слухом. *Той самий* вивід `renderLogSweep` переюзовується аналізатором як еталон деконволюції  $X(t)$ , тож  $X$  байт-ідентичний тому, що випромінює DAC (`FreqRespAnalyzer.java:126`, `getLogSweepBuffer`).

**Обвідна відтворення.** Свіп відтворюється одноразово (без зациклення), видаючи тишу після кінця буфера, тож другий цикл не може згорнутись у вікно захоплення і забруднити деконволюцію (`FreqRespAnalyzer.java:96-106`, `logSweepNext` на `SignalGenerator.java:600`). **Спад Ганна на вході/виході** (вікно Тьюкі) у `SWEEP_FADE_FRACTION_PER_SIDE = 5%` на бік застосовується до меж свіпу, щоб придушити пульсацію спектрального витоку  $1/T$  (бічні пелюстки sinc), яку різкий старт/стоп свіпу інакше впорснув би у  $H(f)$ ;  $5\% \approx 20$  дБ придушення (`sweepEnvelope` на `SignalGenerator.java:625`, довжина спаду `FreqRespCalHelper.sweepFadeSamples` на `:106`).

## 4.2 Часування захоплення та сітки

- **Довжина захоплення** = провідна тиша + свіп + фіксований ½-секундний хвіст (щоб спад імпульсної характеристики пристрою був повністю захоплений до кінця буфера), округлено вгору до цілих секунд (`FreqRespAnalyzer.java:78-82`). Провідна частина дає тракту DAC/ADC влягтися перед першим відліком свіпу.
- **Сітка вихідних частот.**  $N$  геометрично (лог) рознесених точок від `startHz` до `stopHz` (`buildLogSpacedFreqs`, `FreqRespAnalyzer.java:183`):  $f[i] = \exp(\logStart + (\logEnd - \logStart) \cdot i / (N-1))$ . Перемикач часу компіляції `USE_LOG_GRID` (`:61`) може натомість видати лінійно-рознесені точки (`buildLinearFreqs`, `:197`) для споживачів, що хочуть цілих-Гц бінів; продакшн використовує лог. CLI (`FreqRespMode.java:137`) буде ідентичну лог-сітку.
- **Однопрохідне стерео.** Одне відтворення, обидва канали ADC утримано;  $L$  і  $R$  деконволюються на окремих `CompletableFuture`, що поділяють еталонний свіп (`FreqRespAnalyzer.java:127-138`; CLI `FreqRespMode.java:203`).

## 4.3 Деконволюція в частотній області $H = Y/X$

Це серце модуля (`FreqRespCalHelper.computeFromLogSweep`, `FreqRespCalHelper.java:136-370`).



**1. Будова узгоджених буферів X і Y.** Еталон X перебудовується з тим самим спадом Ганна, що застосував плеєр (:153-164) — тож Y і X несуть ідентичні форми меж, а витік старту/стопу скорочується у поділі. Обидва доповнюються нулями до  $M = \text{nextPow2}(\max(yLen, leadIn+sweepLen))$  (:144). Захоплений Y розміщується на відліку 0; еталон X зсунуто на `leadInSamples` для збігу.

**2. Реальне FFT обох, комплексний поділ на бін.** Через `JTransforms DoubleFFT_1D.realForward`. Для кожного півспектрального біна k:  $H = Y/X$  обчислюється як  $H = (Y \cdot \text{conj}(X)) / |X|^2$  —  $hRe = (y_r \cdot x_r + y_i \cdot x_i) / |X|^2$ ,  $hIm = (y_i \cdot x_r - y_r \cdot x_i) / |X|^2$  (:177-191). Біни, де  $|X|^2 = 0$  (поза свіпованою смугою), встановлюються в 0.

**Чому деконволюція над побінним зчитом.** Поділ повного захопленого спектру на повний еталонний спектр відновлює всю передавальну функцію за одну пару FFT, де енергія-на-октаву свіпу дає добрий SNR на кожній частоті — переважно над зчитом окремих FFT-бінів покрокового тону, і це природно дає і фазу, і магнітуду.

**3. Усунення транспортної затримки (годинник/латентність DAC↔ADC).**  $H(f)$  обернено-FFT-ується в імпульсну характеристику (IR). Головний пік IR дає транспортну затримку обходу; суб-відлічна затримка уточнюється **параболічною (квадратичною) інтерполяцією** трьох відліків навколо піка:  $\delta = \text{peak} + 0.5 \cdot (y_- - y_+) / (y_- - 2y_0 + y_+)$  (:201-216). Ця затримка потім усувається як чистий **поворот лінійної фази**, застосований бін-за-біном у частотній області,  $H(k) \cdot e^{+j \cdot 2\pi k \cdot \delta / M}$  (:218-226). Саме так модуль обробляє часовий зсув DAC-відтворення проти ADC-захоплення — і це тримає фізичні  $\pm 180^\circ$  фазові кроки режекторних провалів неушкодженими, замість їх розмазування (обґрунтування у `FreqRespMode.java:67-72`). Примітка: `ClockMismatch.java` (що відображає виміряну-проти-генерованої похибку частоти в ppm назад на головні осцилятори 22.5792/24.576 МГц) використовується **покроково-синусними** ітеративними/FFT-режимами, **не** цим шляхом деконволюції — тут зсув годинника/латентності повністю поглинається членом затримки піка IR.

**4. Опційне часове стробування IR (типово вимкнено).** `USE_IR_GATING=false` (:67). Коли увімкнено, після корекції затримки IR сидить на відліку 0; симетричне вікно зі спадом Ганна напівширини `IR_GATE_LENGTH_SEC` (0.25 с) тримає головний відгук і обнуляє шумовий хвіст, потім переробляє FFT — обмінюючи частотну роздільність ( $\approx 1/\text{gate}$ ) на  $\sim 20$  дБ менше залишкової пульсації (:236-278).

**5. Вибірка H на вихідну сітку.** Для кожної запитаної частоти **лінійна інтерполяція в комплексному спектрі** між двома обхідними FFT-бінами (:284-296): `bin = f/binHz`, змішати `hRe/hIm`, потім `magLin = hypot(re,im)`, `phaseRad = atan2(im,re)`.

**6. Абсолютна нормалізація.** Магнітуди діляться на `amplitudeVrms / adcFsVoltageRms` (захоплений пік ADC у нормованих одиницях для одиничного зворотного зв'язку), тож калібрований одиничного-підсилення зворотний зв'язок читає точно 1.0 (0 дБ) (:298-316). Коментар зазначає, що старіша форма `dacDrivePeak` вносила зсув, коли  $\text{DAC FS} \neq \text{ADC FS}$  (напр., +0.84 дБ) — використання повношкального RMS ADC це виправляє.

## 4.4 Згладжування за Савицьким-Голєєм

**Алгоритм.** Фінальні масиви магнітуди та фази на точку виходу згладжуються фільтром Савицького-Голєя (`USE_SAVGOL_FILTER=true`, вікно 7, порядок 3; `FreqRespCalHelper.java:87-98`, застосовано (:322-344)). SG підганяє поліном низького порядку по ковзному вікну методом найменших квадратів і бере центральний відвід — це придушує шумову пульсацію,

зберігаючи піки/режекторні провали значно краще за ковзне середнє. Коефіцієнти — перший рядок  $(V^T V)^{-1} V^T$  (Вандермонд  $V[i][j] = (i - \text{centre})^j$ ), обернений простим **Гаусом-Жорданом** (`savGolCoefficients` :379, `invertSquareMatrix` :429). Магнітуда згладжується напругу; **фаза згладжується у просторі (sin, cos) і рекомбінується через atan2**, тож завертання  $\pm\pi$  на режекторних провалах її не псують (:327-341). Межі використовують відбиття (`applySavGol` :410). 7-точкове вікно по  $\sim 200$  лог-точках  $\approx 1/12$ -октавне згладжування.

## 4.5 Сховище корекцій .frc (калібрування)

**Призначення.** Скасувати власний відгук вимірювального стенду (звукова карта, кабелі, петлевий шлях DAC→ADC), тож лишається тільки DUT. Калібрування саме є стерео-вимірюванням частотної характеристики, збереженим у файл `.frc/CSV`.

**Формат файлу** (`FreqRespCalHelper.saveCsv` :493, `loadCsv` :538): заголовок-коментар `#` (`kind`, `format_version=6`, `sample_rate`, `sweep range/points`, `DAC drive V RMS`), далі рядки `frequency_hz`, `mag_left_dB`, `mag_right_dB`, `phase_left_deg`, `phase_right_deg`. Магнітуди зберігаються як **відносні дБ** (відношення напруг ADC/DAC), тож плоска смуга пропускання сидить на 0 дБ незалежно від абсолютного масштабування. При завантаженні дБ→лінійне ( $10^{\{dB/20\}}$ ) і град→рад. Розділено комами; застарілі файли з крапкою з комою досі парсяться.

**Модель у пам'яті** (`FreqRespCalibration.java`): паралельні масиви `freqs/magLin/phaseRad`, зростаюча частота, смуга пропускання  $\approx 1.0$ . Стерео-пара = `StereoFreqRespCalibration`.

**Інтерполяція збереженої кривої** (`FreqRespCalHelper.interpolate` :774). Двійковий пошук обхідних точок, потім інтерполяція з **лог-частотою як незалежною змінною**: магнітуда інтерполюється у **дБ** (`db = db0 + (db1 - db0) * t`, потім назад у лінійне), а **фаза через комплексну інтерполяцію одиничного фазора** (змішати `cos/sin`, рекомбінувати з `atan2`), тож завертання  $\pm 180^\circ$  на режекторних провалах інтерполюються правильно. Це ресемплер, використовуваний наскрізно у шляху частотної характеристики — **жодного ресемплінгу Ланцоша тут немає** (Ланцош живе лише у часовому `ScopeLanczos` поданні осцилографа).

**Композиція / володіння сховищем** (`FreqRespCorrectionStore.java`): упорядкований список завантажених калібрувань `Entry` плюс транзитний буфер майстра `direct` (петля сторінки-1). Кілька завантажених файлів **компонуються ланцюжком поділів** по порядку на час малювання. Панель FFT і панель `FreqResp` кожна володіє окремим екземпляром (звичайний об'єкт, не синглтон). Мутації запускають ін'єктований у конструктор `changeNotifier` (GUI публікує подію шини; CLI передає null).

**Час застосування — на час побудови, не на час вимірювання.** Аналізатор повертає **сиру** деконволюцію; калібрування ділиться лише коли крива рендериться (`FreqRespAnalyzer.java`:140-144). Це означає, що заміна калібрувань перетрасовує наявне вимірювання без пересвіпування. У `FreqRespView.applyCurrentCalibration` (`FreqRespView.java`:468) сирий результат клонується, потім кожен запис сховища (і майстер `direct`) ділиться через `divideByStereoCal` (:585) — **лінійно-магнітудний поділ, віднімання фази**, з калом, лог-інтерпольованим на сітку результату. Файли, завантажені з диска, приходять попередньо-скоригованими (`isCalibrationApplied`), тож не діляться знову.

## 4.6 Режекція мережі (інтерполяція в частотній області)

Опційне видалення мережевого гулу і гармонік із показаної кривої ( `FreqRespView.applyMainsNotches` :540 ). Для кожної гармоніки 50/60 Гц до Найквіста кожна точка всередині  $\pm \text{halfWidthHz}$  **замінюється лінійною інтерполяцією (за частотою) магнітуди та фази з двох точок одразу за межами смуги** ( `interpolateAcrossBand` :555 ). Напівширина масштабується з довжиною FFT деконволюції, `halfWidth = 1.2 * sampleRate / fftSize` ( :509 ), тож завжди охоплює фіксоване число ширин біна ( $\approx \pm 8$  Гц при 48 кГц / 1.1 с). Це інтерполяція на час побудови по забруднених бінах, а не часовий гребінчастий фільтр.

## 4.7 Крива RIAA фоно-корекції

`RiaaCurve.java` обчислює криву фоно-еквалізації, накладену на / відняту з вимірювання (для тестування фоно-передпідсилювачів).

**Математика.** Передавальна функція запису (кодування) в s-області —  $H_{\text{record}}(s) = (s \cdot T_1 + 1)(s \cdot T_3 + 1) / (s \cdot T_2 + 1)$  зі стандартними сталими часу  **$T_1 = 3180$  мкс** (50.05 Гц),  **$T_2 = 318$  мкс** (500.5 Гц),  **$T_3 = 75$  мкс** (2122 Гц), плюс опційний IEC  **$T_4 = 7950$  мкс** (20.02 Гц субзвуковий ФВЧ). Квадрат магнітуди обчислюється напям ( `recordDb` :92 ):  $|H|^2 = (1 + \omega^2 T_1^2)(1 + \omega^2 T_3^2) / (1 + \omega^2 T_2^2)$ . Крива **відтворення (декодування)** — канонічна «крива RIAA», що падає з +20 дБ @ 20 Гц до −20 дБ @ 20 кГц — це заперечена крива запису (прапор `reverse`). Усе **нормовано на 0 дБ при 1 кГц** ( `REF_HZ` , :64 ) відніманням `recordDb(1 кГц)`.

**Поправка IEC.** Коли увімкнено, магнітуда запису **ділиться** (не множиться) на однополюсний ФВЧ  $|H_P|^2 = \omega^2 T_4^2 / (1 + \omega^2 T_4^2)$  ( :98-101 ) — математично обернено до того, як IEC входить у тракт відтворення, тримаючи запис<sup>-1</sup>·відтворення = тотожність, тож плоска петля досі читає 0 дБ скрізь навіть з увімкненим IEC.

## 4.8 Накладка RIAA, траса порівняння/різниці, прив'язування

- **Накладка** ( `FreqRespView.drawRiaaOverlay` :885 ). Аналітична крива RIAA вибирається кожні 5 px по plot і малюється як пунктирний еталон. Вона **прив'язана при 1 кГц до значення 1 кГц вимірюної траси** ( `riaaAnchorDb` :907 ), тож еталон вишиковується з даними.
- **Режим порівняння (віднімання)** ( `getCompareDiff` :976 , `drawCompareTrace` :933 ). Обчислює на точку ( `measuredDb - RiaaCurve.evalDb` ), тобто вимірювання мінус цільова крива RIAA, тож ідеальний фоно-передпідсилювач читає плоску лінію 0 дБ. Різниця потім **згладжується ковзним середнім у лог-частотному просторі по вікну 1/W-октави** (  $W = \text{налаштування}$ , напівширина  $\log_{10}(2)/(2W)$ , двовказівникова  $O(n)$  поточна сума, :1013-1041 ) — обрано над індексним вікном, бо на 192k-точковому лог-світі фіксоване число індексів покриває дико різні смуги на низьких проти високих частот. Згладжена крива **прив'язана до 0 дБ при 1 кГц** відніманням свого інтерпольованого значення 1 кГц ( `valueAt1kHz` :1047 / :1380 ). Min/max по 20 Гц–25 кГц живлять екранну таблицю. Один кешований об'єкт `CompareDiff` живить намальовану трасу, таблицю min/max і  $\Delta$ -зчит перехрестя, тож усі три узгоджені за побудовою.
- **Авто-налаштування** ( `autoSetupCompare` :1086 , `autoSetupMagnitudeRange` :660 ). Підганяє подання до даних: вертикальне вікно = min/max даних + 10% запас (режим порівняння тримає +20 дБ запас над піком,  $\geq 2$  дБ мінімальний прогін, центрований на середині сигналу, коли природна підгонка тісніша).

## 4.9 Живе вимірювання під час світу

`FreqRespLiveMeter.java` — компактний графік «рівень входу проти часу», показаний у зайнятій оболонці, поки біжить свіп (у стилі REW). На блок захоплення він отримує час, що минув + лінійний RMS; переводить у  $\text{dbFs} = 20 \cdot \log_{10}(\text{rms})$ , затиснутий до осі  $-100..0$  дБ. Вхідний RMS спершу проходить через **експоненційне ковзне середнє** (`EMA_ALPHA = 0.95`, : 88 / : 147) — на низьких частотах кожен  $\sim 1024$ -відлічний блок тримає лише частковий цикл, тож сирий RMS дико коливається у гребінку зубців; ЕМА колапсує це у стабільну амплітудну обвідну. Малюється як полілінія; понад `MAX_POINTS` децимується на 2, щоб лишатись чуйним на довгих захопленнях.

#### 4.10 Побудова (лише freq-resp-специфічні частини)

Подання розширює спільний рушій `AbstractFreqDomainView/AbstractMeasurementView` (§3.16), що надає **лог-частотну x-вісь**, «**гарну-поділкову**» **дБ y-вісь** (`AxisSpec.linearNice` округлює до кроків  $1/2/2.5/5 \times 10^n$ ), перетворення `freqToX` (лог)/`dbToY`, `paintPolyline` з відсіканням і кеш малювання статичного шару. `Freq-resp`-специфічне:

- **Налаштування осей** (`FreqRespView.onPaint` :805-814): `AxisSpec.log(freqMin, freqMax)` для x; `linearNice(magBot, magTop, ...)` у дБ для лівої y; і опційна **фіксована  $\pm 180^\circ$  фазова права-вісь** (`AxisSpec.linear(-180, 180, 8)`), показана лише коли фазову трасу увімкнено.
- **Фазова траса** малюється пунктирною полілінією, `phaseToY` відображає градуси на фіксовану  $\pm 180^\circ$  вісь (`phaseToYFraction` затискає потім відображає, `FreqRespFormat.java`: 132).
- **Координатна/форматна математика** (`FreqRespFormat.java`): `freqToXFraction/xFractionToFreq` ( $\log_{10}$ ), `linToDb` (повертає  $-300$  для  $\leq 0$ , щоб уникнути NaN), форматери зчиту дБ/фази зі значущими цифрами.
- **Зчит перехрестя** (`drawCrosshair` :1287) інтерполює активну трасу (лог-частота, лінійно-дБ) на курсорі — а в режимі порівняння інтерполює *той самий* кешований масив згладженої-різниці, тож  $\Delta$ -зчит збігається з намальованою кривою. Частота курсора відсікається на `freqRespNyquistFraction`  $\times$  `sampleRate`.
- **Зум/панорамування**: лог-частотний зум навколо курсора (:1485, геометричний), лінійно-дБ зум/панорамування магнітуди, усе збережено у `Preferences` і опубліковано на `FREQRESP_RANGE_CHANGED`.

Шлях CLI (`FreqRespMode.java`) рендерить еквівалентний `JFreeChart` PNG із кастомним `LogAxis`, чий `refreshTicks` ставить основні на  $\{1, 2, 3, 5, 7\} \times 10^n$ , магнітуду на лівій осі та розгорнуту фазу (пунктир) на правій осі.